

Langgraph Notes

Module 1 — LangGraph Fundamentals

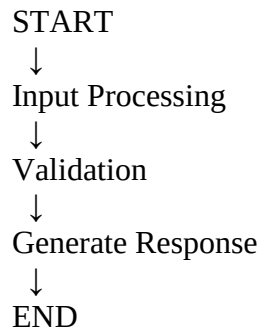
1. What is LangGraph?

LangGraph is a framework for building **stateful, multi-step applications** using a graph-based architecture.

It allows you to represent an application's workflow as a **graph**, where:

- **Nodes** represent operations/tasks.
- **Edges** define how execution moves between nodes.
- **State** stores information shared between different steps.
- **START** represents where execution begins.
- **END** represents where execution finishes.

A simple workflow can look like:



The important idea is that LangGraph doesn't just execute a sequence of functions. It allows you to build workflows where execution can:

- move forward
- make decisions
- repeat steps
- maintain state
- branch into different paths
- pause and resume

Simple definition for interview

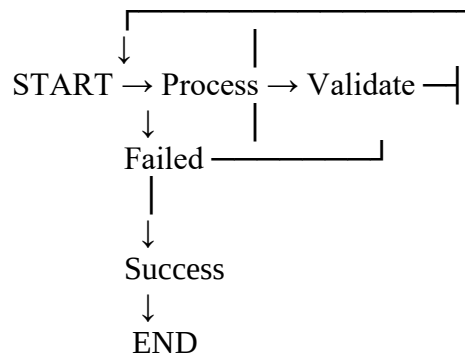
LangGraph is a graph-based framework for building stateful, controllable, and multi-step applications where nodes perform operations, edges control execution flow, and state maintains information throughout the workflow.

2. Why LangGraph?

Traditional applications often follow a simple sequence:

Step 1 → Step 2 → Step 3 → Step 4

But real applications can require more complex behavior:



For example:

1. Receive user input
2. Analyze input
3. Validate result
4. If invalid → try again
5. If valid → continue
6. Store information
7. Generate final result

LangGraph provides explicit control over this workflow.

Main reasons to use LangGraph

- Stateful workflows
 - Multi-step processing
 - Conditional execution
 - Loops/cycles
 - Branching
 - Human-in-the-loop workflows
 - Long-running workflows
 - Checkpointing/persistence
 - Complex application control flow
-

3. LangGraph vs LangChain

This is a **very common interview question**.

LangChain

LangChain provides components for building LLM applications.

For example:

Prompt
↓
LLM
↓
Parser
↓
Output

It provides components such as:

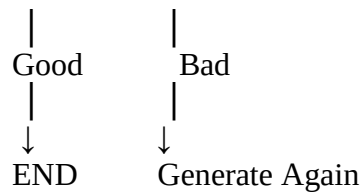
- Models
- Prompts
- Output parsers
- Retrievers
- Tools
- Document loaders
- Embeddings
- Vector stores

LangGraph

LangGraph focuses more on **workflow orchestration and state management**.

Example:

START
↓
Retrieve Documents
↓
Generate Answer
↓
Check Answer
↓
└──────────┘



Comparison

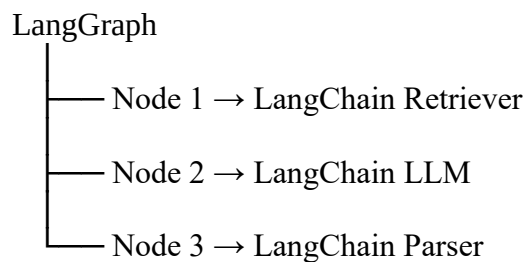
LangChain	LangGraph
Provides LLM application components	Provides graph-based workflow orchestration
Chains/components	Nodes and edges
Useful for sequential pipelines	Useful for complex workflows
Less focus on persistent state	Strong state management
Mostly linear patterns can be simple	Supports branches and cycles naturally
Components can be used independently	Coordinates components into workflows

Important

LangGraph does **not** replace LangChain.

You can use LangChain components **inside** LangGraph nodes.

For example:



Interview answer

LangChain provides components for building LLM applications, while LangGraph provides a graph-based orchestration layer for building stateful, multi-step workflows. LangChain components can be used inside LangGraph nodes.

4. LangGraph vs Traditional Workflows

A traditional workflow is often linear:

$A \rightarrow B \rightarrow C \rightarrow D$

For example:

Read File
↓
Process Data
↓
Save Data

But LangGraph can represent:

Conditional workflow

A
↓
B
↓
Is valid?
↙ ↘
Yes No
↓ ↓
C D

Cyclic workflow

$A \rightarrow B \rightarrow C$
↑ ↓
└──┘

Stateful workflow

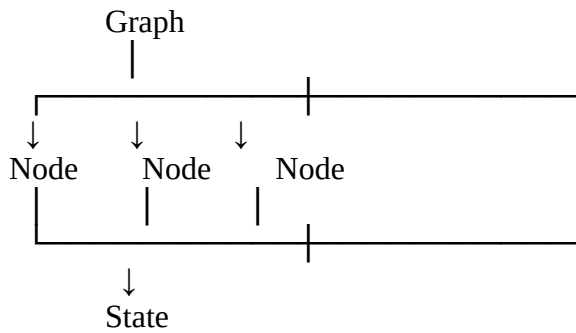
State
↓
 $A \rightarrow$ modifies state
↓
 $B \rightarrow$ reads state
↓
 $C \rightarrow$ modifies state
↓
 $D \rightarrow$ reads state

This makes LangGraph useful when workflow execution depends on the current state.

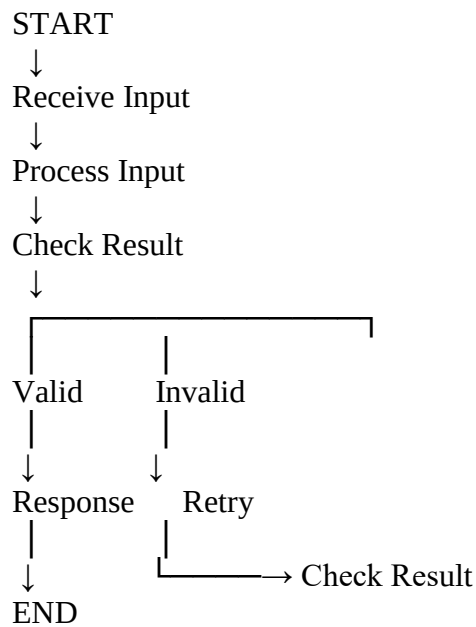
5. Graph-Based Application Architecture

LangGraph represents an application as a **graph**.

The basic architecture is:



A more realistic example:



6. What is a Graph?

A **graph** is a collection of:

- Nodes
- Edges

In LangGraph, the graph represents the overall application workflow.

Example:

START → A → B → C → END

Here:

A, B, C = Nodes

→ = Edges

The graph determines how your application executes.

7. What is a Node?

A **node** is a function that performs some operation.

For example:

```
def process_input(state):  
    return {"result": "processed"}
```

This function can become a LangGraph node.

A node might:

- Call an LLM
- Query a database
- Call an API
- Process data
- Validate information
- Execute business logic
- Modify state

Example:

Node 1 → Get User Input

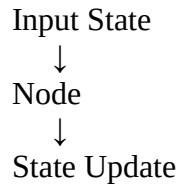
Node 2 → Process Input

Node 3 → Validate Input

Node 4 → Generate Output

Important

A node generally:



8. What is an Edge?

An **edge** determines how execution moves from one node to another.

Example:

$A \rightarrow B$

means:

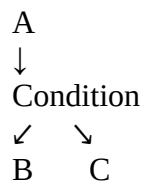
After node A executes, execution moves to node B.

Edges can be:

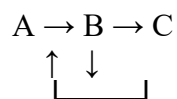
Normal edge

$A \rightarrow B$

Conditional edge



Cyclic edge



9. What is State?

State is the shared information that flows through the graph.

This is one of the **most important LangGraph concepts**.

Imagine a workflow:

```
START
↓
Read Input
↓
Process Input
↓
Generate Output
↓
END
```

The nodes need to share information.

For example:

```
{
  "user_input": "Explain Python",
  "result": "...",
  "status": "success"
}
```

This information is called **state**.

Simple definition

State is the structured data that stores the current information of a LangGraph workflow and is passed between nodes.

10. StateGraph

StateGraph is used to create a graph whose nodes operate on a shared state.

Conceptually:

```
State Schema
↓
StateGraph
↓
Nodes + Edges
↓
Compile
```

Prathamesh Arvind Jadhav

↓
Runnable Graph

Example:

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph
```

```
class State(TypedDict):
    message: str
```

```
graph = StateGraph(State)
```

Here:

State = structure of workflow data
StateGraph = graph built around that state

11. START and END

LangGraph provides special graph markers:

```
START
↓
Node A
↓
Node B
↓
END
```

START

START indicates where graph execution begins.

END

END indicates that graph execution has completed.

Example:

```
from langgraph.graph import START, END
```

```
builder.add_edge(START, "node_a")
builder.add_edge("node_a", END)
```

Execution:

```
START
↓
node_a
↓
END
```

12. Graph Execution Flow

A typical LangGraph workflow follows this lifecycle:

```
1. Define State
↓
2. Create StateGraph
↓
3. Add Nodes
↓
4. Add Edges
↓
5. Compile Graph
↓
6. Invoke Graph
↓
7. Nodes execute
↓
8. State gets updated
↓
9. Execution continues
↓
10. END
```

Example:

```
Input State
↓
START
↓
Node A
↓
State Update
↓
Node B
↓
State Update
```

↓
END

13. Directed Graphs

A **directed graph** is a graph where edges have a direction.

Example:

$A \rightarrow B$

This means:

$A \rightarrow B$

but not automatically:

$B \rightarrow A$

LangGraph workflows use directed connections.

Example:

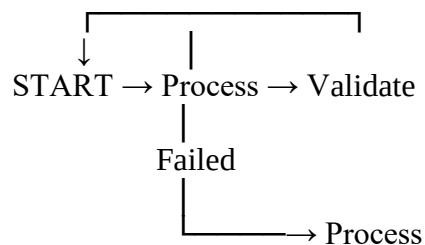
$\text{START} \rightarrow \text{Process} \rightarrow \text{Validate} \rightarrow \text{END}$

The direction determines execution flow.

14. Cyclic Graphs

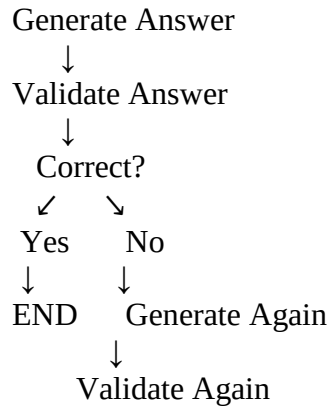
A cyclic graph contains a path that can return to an earlier node.

Example:



This is useful when a workflow needs to repeat.

Example:



This is one of the major differences between simple chains and graph-based workflows.

15. Stateful Workflows

A **stateful workflow** remembers information while execution progresses.

Example:

```
State = {  
  "question": "...",  
  "documents": [...],  
  "answer": "...",  
  "score": 0.85  
}
```

Node 1:

```
question  
↓  
retrieve documents
```

State becomes:

```
question  
documents
```

Node 2 reads that state:

```
question + documents  
↓
```

generate answer

State becomes:

question
documents
answer

Node 3 can then use:

question
documents
answer

This allows information to persist across workflow steps.

16. Deterministic vs Dynamic Workflows

Deterministic Workflow

The execution path is predictable.

$A \rightarrow B \rightarrow C \rightarrow D$

Every execution follows the same path.

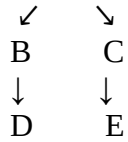
Example:

Load Data
↓
Clean Data
↓
Transform Data
↓
Save Data

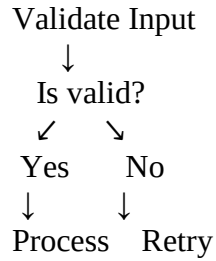
Dynamic Workflow

The execution path depends on conditions or state.

A
↓
Condition



For example:



LangGraph is especially useful for these dynamic workflows.

17. When to Use LangGraph?

LangGraph is useful when your application has:

1. Multiple workflow steps

$A \rightarrow B \rightarrow C \rightarrow D$

2. Conditional branching

$A \rightarrow B$

 $C \quad D$

3. Loops

$A \rightarrow B \rightarrow C$

4. Shared state

Multiple nodes need access to the same information.

5. Long-running workflows

Workflows that may execute for a long time.

6. Human interaction

A workflow can pause for human input and continue later.

7. Persistence

You need to save workflow state and potentially resume execution.

8. Complex orchestration

Multiple components need coordinated execution.

18. When NOT to Use LangGraph?

Don't automatically use LangGraph for every application.

If your workflow is simply:

```
Input
↓
Process
↓
Output
```

a normal Python function may be enough.

For example:

```
def process(data):
    result = clean(data)
    result = transform(result)
    return save(result)
```

You may not need a graph.

Avoid unnecessary LangGraph when:

- Workflow is very simple
- No shared state is required
- No branching is required
- No loops are required
- A normal function or simple pipeline is easier

Rule of thumb

Use LangGraph when workflow complexity and state management justify graph-based orchestration.

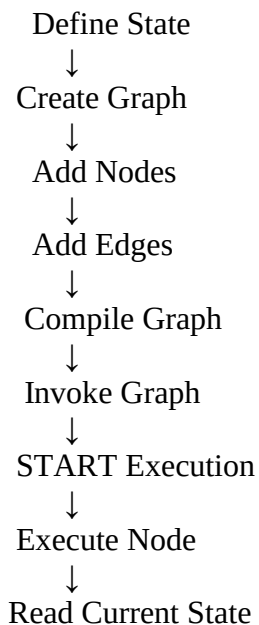
20. LangGraph Core Components

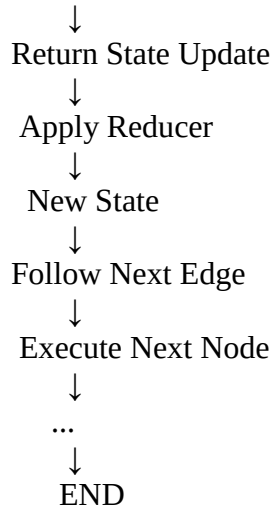
The most important components are:

Component	Purpose
StateGraph	Builds a stateful graph
State Schema	Defines structure of state
Node	Performs an operation
Edge	Defines execution flow
Conditional Edge	Chooses next node dynamically
START	Entry point
END	Exit point
Reducer	Determines how state updates are combined
Checkpoint	Enables persistence/checkpointing
Compile	Converts graph definition into executable graph

21. LangGraph Execution Lifecycle

The high-level lifecycle is:





This lifecycle is extremely important for understanding the framework.

Module 2 — State Management

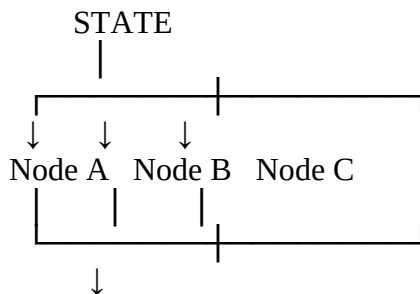
22. What is State?

State represents the current information available to the graph.

For example:

```
{
  "name": "Prathamesh",
  "question": "What is LangGraph?",
  "answer": "",
  "status": "processing"
}
```

Different nodes can read and update this information.



Updated State

23. Why is State Required?

Imagine three nodes:

Node A → Node B → Node C

Node A calculates something.

Node B needs the result from Node A.

Node C needs the result from Node B.

Without shared state, you need to manually pass data between every function.

State provides a common structure:

```
Node A
↓
updates state
↓
Node B
↓
reads state
↓
updates state
↓
Node C
```

24. Defining State

LangGraph commonly uses Python type definitions such as TypedDict.

Example:

```
from typing_extensions import TypedDict

class State(TypedDict):
    name: str
    message: str
```

This defines the structure:

State

```
├── name
└── message
```

Example state:

```
{
  "name": "Prathamesh",
  "message": "Hello"
}
```

25. What is TypedDict?

TypedDict comes from Python's typing system.

It allows you to define the expected keys and types of a dictionary.

Example:

```
class State(TypedDict):
    name: str
    age: int
```

Valid:

```
{
  "name": "Prathamesh",
  "age": 22
}
```

Conceptually:

```
TypedDict
↓
Defines expected structure
↓
State Schema
```

Important

TypedDict mainly provides **type information**. It is not the same as a runtime validation model like Pydantic.

26. State Schema

A **state schema** defines what information your graph state contains.

Example:

```
class State(TypedDict):  
    question: str  
    answer: str  
    score: float
```

Schema:

```
State  
├── question → str  
├── answer  → str  
└── score   → float
```

Then:

```
graph = StateGraph(State)
```

tells LangGraph that this is the state structure used by the graph.

27. State Fields

Each key inside your state is a **state field**.

Example:

```
class State(TypedDict):  
    question: str  
    answer: str  
    documents: list
```

Fields:

```
question  
answer  
documents
```

Each field stores a particular piece of workflow information.

28. Reading State

A node receives state and can read values from it.

Example:

```
def process(state):  
    question = state["question"]  
  
    print(question)  
  
    return {}
```

Flow:

```
State  
↓  
Node  
↓  
Read question
```

29. Updating State

A node can return an update to state.

Example:

```
def process(state):  
    return {  
        "answer": "LangGraph is a graph-based framework."  
    }
```

The node doesn't necessarily need to return the entire state.

It can return only the field it wants to update.

30. Partial State Updates

This is an important concept.

Suppose state is:

```
{
```

Prathamesh Arvind Jadhav

```
"name": "Prathamesh",  
"age": 22,  
"city": "Pune"  
}
```

A node can return:

```
{  
  "age": 23  
}
```

Conceptually:

```
Old State  
{  
  name: Prathamesh  
  age: 22  
  city: Pune  
}
```

↓

```
Update  
{  
  age: 23  
}
```

↓

```
New State  
{  
  name: Prathamesh  
  age: 23  
  city: Pune  
}
```

The node is providing a **partial update**, rather than recreating the entire state.

31. State Immutability Concept

Think of LangGraph state updates conceptually as:

```
Old State  
↓
```

Node produces update
↓
Reducer applies update
↓
New State

Rather than thinking:

Node directly modifies global state

think:

Node
↓
returns update
↓
LangGraph applies update
↓
new state

This makes workflow execution easier to reason about.

32. What is a Reducer?

A **reducer** determines how a node's update should be combined with the existing state value.

This becomes particularly important when multiple updates affect the same state field.

Consider:

```
class State(TypedDict):  
    messages: list
```

Suppose the current state is:

```
messages = ["Hello"]
```

A node returns:

```
messages = ["How are you?"]
```

Should the result be:

```
["How are you?"]
```

or:

["Hello", "How are you?"]

The reducer determines how the update is handled.

Simple definition

A reducer defines how new values returned by nodes are combined with the existing value of a state field.

33. Default Reducer Behavior

For a state field without a special reducer, the new value generally **replaces** the existing value.

Example:

Old:

answer = "Old answer"

Update:

answer = "New answer"

Result:

answer = "New answer"

Conceptually:

old value



new update



replacement

This is useful for fields where only the latest value matters.

34. Custom Reducers

Sometimes you don't want replacement.

You want to combine values.

For example:

Existing:

[1, 2]

Update:

[3, 4]

Desired:

[1, 2, 3, 4]

A reducer can define this behavior.

35. Annotated

Python's Annotated allows additional metadata to be associated with a type.

In LangGraph, it is commonly used to attach a reducer to a state field.

Example:

```
from typing import Annotated
import operator
```

```
class State(TypedDict):
    numbers: Annotated[list[int], operator.add]
```

Here:

```
list[int]
+
operator.add
```

means that operator.add is used as the reducer for that field.

36. operator.add

Python's operator.add represents addition.

For lists, it effectively combines them:

[1, 2] + [3, 4]

produces:

[1, 2, 3, 4]

Therefore:

`Annotated[list[int], operator.add]`

can be used when you want list updates to accumulate.

Conceptually:

Existing list

+

New list

↓

Combined list

37. Example of Reducer

Consider:

```
from typing import Annotated
from typing_extensions import TypedDict
import operator
```

```
class State(TypedDict):
    numbers: Annotated[list[int], operator.add]
```

Initial state:

```
{
    "numbers": [1, 2]
}
```

Node returns:

```
{
    "numbers": [3, 4]
}
```

Reducer behavior:

```
[1, 2] + [3, 4]
↓
```

[1, 2, 3, 4]

Without the reducer, the normal update behavior would not accumulate the values this way.

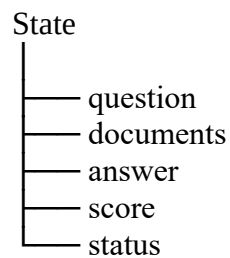
38. Multiple State Fields

A state can contain many fields.

Example:

```
class State(TypedDict):  
    question: str  
    documents: list[str]  
    answer: str  
    score: float  
    status: str
```

Visualize it as:



Different nodes can update different fields.

Retriever Node



documents

Generator Node



answer

Validator Node



score

status

39. Messages State

Many LLM applications need to maintain a conversation.

For example:

User: What is Python?

AI: Python is a programming language.

User: What is a list?

AI: A list is a collection...

The conversation can be represented as a list of messages.

LangGraph provides a convenient state type called:

MessagesState

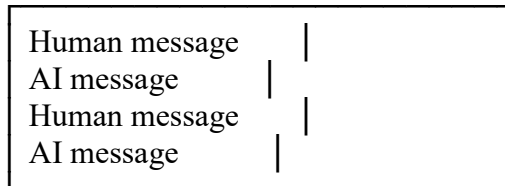
It is designed for workflows that need to work with message-based state.

Conceptually:

MessagesState



messages



40. Why MessagesState?

Instead of manually defining a messages field, you can use a prebuilt state structure designed for message-based workflows.

Conceptually:

```
from langgraph.graph import MessagesState
```

Then:

```
class State(MessagesState):
```

```
...
```

or use MessagesState directly where appropriate.

The exact structure and message handling are managed by LangGraph's message-state functionality.

Interview definition

MessagesState is a prebuilt LangGraph state schema designed to maintain a list of messages, making it convenient for conversational and message-based workflows.

41. State Validation

State validation means ensuring that the data follows the expected state structure and types.

For example:

```
class State(TypedDict):
    name: str
    age: int
```

Expected:

```
{
    "name": "Prathamesh",
    "age": 22
}
```

The state schema communicates the expected structure to developers and tooling.

Important distinction

TypedDict provides **static/type-level structure**, but it should not be confused with full runtime validation.

If strict runtime validation is required, tools such as Pydantic can be considered depending on the application's design.

42. State Transformation

Nodes can transform state.

Example:

Input State

Prathamesh Arvind Jadhav

↓
Node
↓
Transform
↓
Updated State

Suppose:

```
{  
  "name": "prathamesh"  
}
```

A node could return:

```
{  
  "name": "PRATHAMESH"  
}
```

The workflow has transformed the state.

Another example:

Raw Input
↓
Clean Data
↓
Processed Data
↓
Validated Data

43. State Lifecycle

State changes throughout graph execution.

Example:

Initial state

```
{  
  "question": "What is LangGraph?"  
}
```

After Node 1

```
{  
  "question": "What is LangGraph?",  
  "documents": [...]  
}
```

After Node 2

```
{  
  "question": "What is LangGraph?",  
  "documents": [...],  
  "answer": "..."  
}
```

After Node 3

```
{  
  "question": "What is LangGraph?",  
  "documents": [...],  
  "answer": "...",  
  "score": 0.92  
}
```

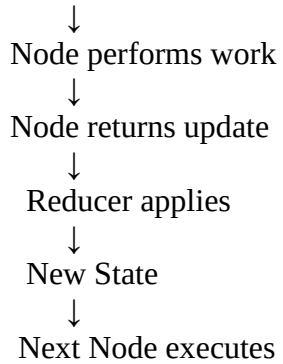
So:

Initial State
↓
Node 1
↓
Updated State
↓
Node 2
↓
Updated State
↓
Node 3
↓
Final State

44. The Most Important State Flow

Memorize this:

State
↓
Node reads State



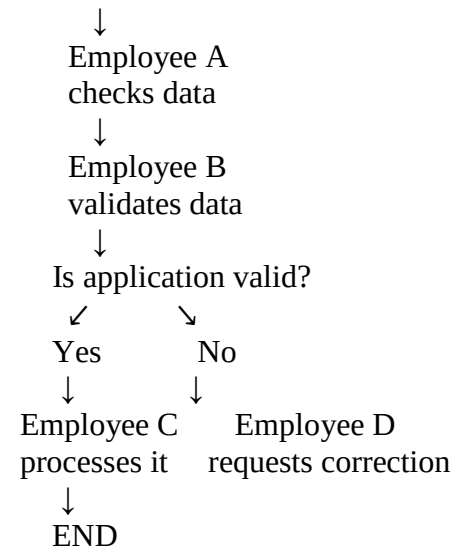
This is the foundation of LangGraph state management.

45. Complete Mental Model

Think about LangGraph like a **company workflow**.

Suppose:

Customer submits application



Now map this to LangGraph:

Real World	LangGraph
Workflow	Graph
Employee/task	Node
Handoff	Edge
Shared application information	State

Real World	LangGraph
Decision	Conditional edge
Rechecking	Cycle
Start	START
Completion	END

This mental model makes LangGraph much easier to understand.

Module 3 — Nodes and Edges

Goal

Learn how to construct LangGraph workflows using:

Nodes = What the application does

Edges = Where execution goes next

The basic idea is:

```
START
↓
Node A
↓
Node B
↓
Node C
↓
END
```

1. What is a Node?

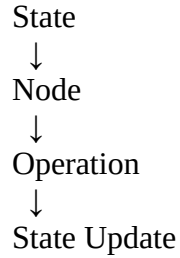
A **node** is a function that performs a specific operation in a LangGraph workflow.

For example:

```
def process_data(state):
    # perform some operation
    return {"result": "processed"}
```

You can add this function to a graph as a node.

Conceptually:



A node can perform almost any application logic:

- Call an LLM
- Retrieve documents
- Query a database
- Call an API
- Validate data
- Transform data
- Execute business logic
- Generate a response

Interview definition

A node is a function or operation in LangGraph that receives state, performs some work, and returns a state update.

2. Node Function

A node is usually represented by a Python function.

Example:

```
def node_a(state):  
    return {  
        "message": "Hello"  
    }
```

Then the function is registered with the graph:

```
builder.add_node("node_a", node_a)
```

Here:

```
"node_a" → Node name  
node_a  → Python function
```

3. Node Input

A node generally receives the current graph state.

Example:

```
def process_user(state):  
    name = state["name"]  
  
    print(name)  
  
    return {}
```

If the state is:

```
{  
  "name": "Prathamesh"  
}
```

then the node can access:

```
state["name"]
```

Flow:

```
Current State  
  ↓  
Node Function  
  ↓  
Read State
```

4. Node Output

A node normally returns a **state update**.

Example:

```
def generate_answer(state):  
    return {  
        "answer": "LangGraph is a graph-based framework."  
    }
```

If the existing state is:

```
{
```

```
"question": "What is LangGraph?"  
}
```

the node returns:

```
{  
  "answer": "LangGraph is a graph-based framework."  
}
```

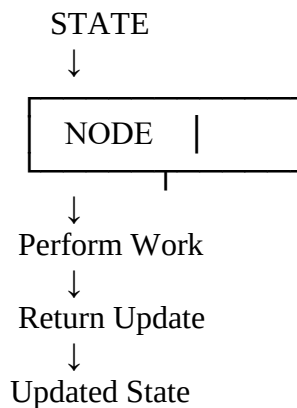
The graph applies that update to the state.

Conceptually:

```
Old State  
↓  
Node  
↓  
State Update  
↓  
Updated State
```

5. Node Input → Processing → Output

The most important node mental model is:



For example:

```
def calculate(state):  
    result = state["a"] + state["b"]  
  
    return {  
        "result": result  
    }
```

6. Sync Nodes

A **synchronous node** is a normal Python function.

Example:

```
def process(state):
    result = state["value"] * 2

    return {
        "result": result
    }
```

It executes synchronously.

Use sync nodes when the operation itself is synchronous.

7. Async Nodes

An **asynchronous node** uses `async def`.

Example:

```
async def process(state):
    result = await some_async_operation()

    return {
        "result": result
    }
```

Async nodes are useful when working with asynchronous operations such as:

- Async API calls
- Async LLM calls
- Async database operations
- Network requests

Conceptually:

Sync Node

Node → Work → Result

versus:

Async Node

Node → await operation → Result

Important

Don't use async simply because you are using LangGraph. Use it when the underlying work benefits from asynchronous execution.

8. Multiple Nodes

A graph can contain many nodes.

Example:

```
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_node("node_c", node_c)
```

Then connect them:

```
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", "node_c")
builder.add_edge("node_c", END)
```

Workflow:

```
START
↓
Node A
↓
Node B
↓
Node C
↓
END
```

9. Node Naming

Each node should have a meaningful name.

Good:

```
builder.add_node("retrieve_documents", retrieve_documents)
builder.add_node("generate_answer", generate_answer)
builder.add_node("validate_answer", validate_answer)
```

Avoid unclear names:

```
builder.add_node("x", func1)
builder.add_node("y", func2)
```

Good naming becomes especially important when debugging large graphs.

Best practice

Use names that describe **what the node does**.

For example:

```
retrieve_documents
validate_input
generate_response
save_result
```

10. Node Dependencies

Nodes can depend on information produced by previous nodes.

Example:

```
Node A
↓
creates documents
↓
Node B
↓
uses documents
```

State acts as the communication mechanism.

Example:

```
def retrieve(state):
    documents = ["doc1", "doc2"]
```

```
return {  
  "documents": documents  
}
```

Then:

```
def generate(state):  
  documents = state["documents"]  
  
  return {  
    "answer": "Answer based on documents"  
  }
```

So:

```
Node A  
↓  
State.documents  
↓  
Node B
```

11. Node Execution

A node executes when the graph reaches that node according to the graph's execution rules.

For a simple graph:

```
START  
↓  
A  
↓  
B  
↓  
C  
↓  
END
```

execution is:

1. START
2. A executes
3. A returns update
4. State is updated
5. B executes
6. B returns update

7. State is updated
 8. C executes
 9. C returns update
 10. END
-

12. What is an Edge?

An **edge** defines the relationship or execution path between nodes.

Example:

$A \rightarrow B$

means:

After A, execution can proceed to B.

Interview definition

An edge defines the control flow of a LangGraph by determining which node or nodes can execute next.

13. Normal Edges

A normal edge connects one node to another.

$A \rightarrow B$

In LangGraph:

```
builder.add_edge("node_a", "node_b")
```

This means:

```
node_a
  ↓
node_b
```

14. add_edge()

`add_edge()` is used to connect graph nodes.

Basic syntax:

```
builder.add_edge(source, destination)
```

Example:

```
builder.add_edge("node_a", "node_b")
```

Workflow:

Node A



Node B

15. START Edge

You need to tell LangGraph where the workflow begins.

Use:

```
builder.add_edge(START, "node_a")
```

Workflow:

START



Node A

START is a special graph entry point.

16. END Edge

You can connect a node to END.

```
builder.add_edge("node_c", END)
```

Workflow:

Node C



END

END indicates that execution has finished.

17. Complete Sequential Workflow

Example:

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

```
class State(TypedDict):
    message: str
```

```
def node_a(state):
    return {
        "message": "Hello"
    }
```

```
def node_b(state):
    return {
        "message": state["message"] + " World"
    }
```

```
builder = StateGraph(State)
```

```
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
```

```
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", END)
```

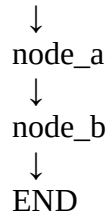
```
graph = builder.compile()
```

```
result = graph.invoke({
    "message": ""
})
```

```
print(result)
```

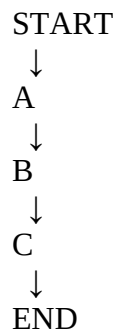
Architecture:

START

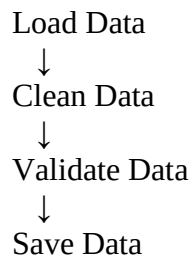


18. Sequential Execution

A sequential workflow executes one step after another.



For example:

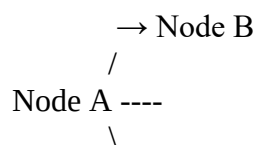


Each node depends on the previous workflow step.

19. Multiple Outgoing Edges

A node can have multiple outgoing paths.

Example:



→ Node C

This is useful for branching.

However, there is an important distinction:

Multiple outgoing edges

Can result in multiple destinations being scheduled.

Conditional edges

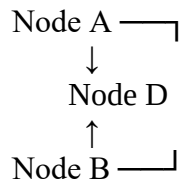
Are used when you want a routing decision based on state/result.

For decision-making workflows, **conditional edges** are generally the appropriate mechanism.

20. Multiple Incoming Edges

Multiple nodes can point to the same destination.

Example:



This can be useful when different branches eventually converge.

For example:



21. How Does LangGraph Determine Execution Order?

LangGraph uses the **graph structure and edges** to determine which nodes are eligible to execute.

Example:

$A \rightarrow B \rightarrow C$

means:

A first
↓
B second
↓
C third

For branching:

→ B
/
A
\
→ C

the graph structure allows execution to proceed through the relevant outgoing paths.

The important point is:

Execution order is determined by graph connectivity and the state-dependent routing rules, not simply by the order in which nodes were written in Python.

22. Branching Workflows

A branching workflow has multiple possible paths.

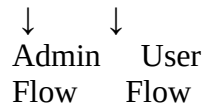
Example:

START → Node A → Node D → END
┌→ Node B ─┐
└→ Node C ─┘

Node A decides which path should execute.

Example:

Check User
↓
Is Admin?
↙ ↘
Yes No

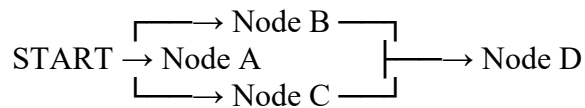


This is usually implemented using conditional routing.

23. Parallel Execution

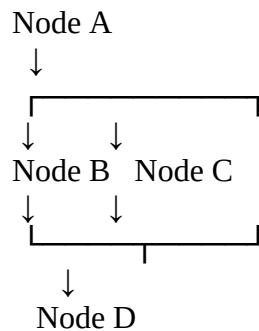
LangGraph can support workflows where multiple branches become ready to execute.

Example:



Node B and Node C are independent branches.

Conceptually:



Important interview point

Parallel execution is different from simply having multiple nodes in a graph.

The graph must have independent branches that can be scheduled without one branch depending on the other.

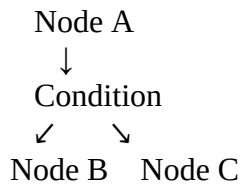
When branches converge, state update behavior and reducers become important.

24. Conditional Routing

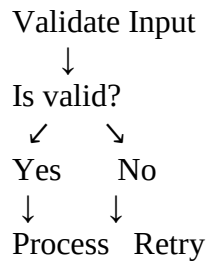
Conditional routing means:

Choose the next node based on some condition.

Example:



For example:

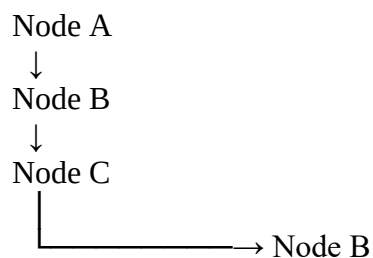


This is covered in depth in Module 4.

25. Cyclic Workflows

A cyclic workflow contains a loop.

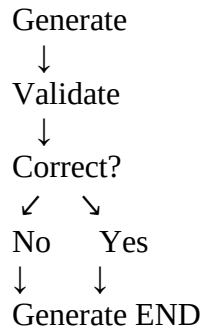
Example:



This can be useful for:

- Retry logic
- Validation
- Iterative processing
- Refinement
- Repeated evaluation

Example:



26. Node vs Edge

This is one of the most important interview questions.

Node	Edge
Performs an operation	Controls flow
Usually a function	Connection between graph elements
Processes state	Determines what executes next
Contains business logic	Contains routing/control logic
Example: generate answer	Example: $A \rightarrow B$

Easy way to remember

Node = What to do

Edge = Where to go

27. Module 3 Complete Mental Model

Think of a railway system:

Station = Node

Railway track = Edge

Train = Execution

Information carried by train = State

For example:

START



Station A



Station B

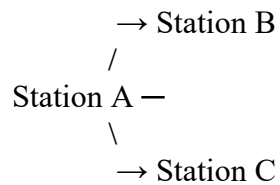


Station C



END

A branch:



The track determines where the train can go.

Similarly:

Edges determine where graph execution can go.

Module 4 — Conditional Routing & Graph Control

Goal

Learn how LangGraph can **make decisions and control workflow execution**.

The key concept is:

Node



Routing Decision

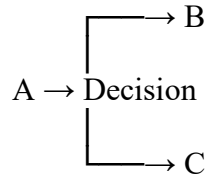


Choose Next Node

Instead of:

$A \rightarrow B \rightarrow C$

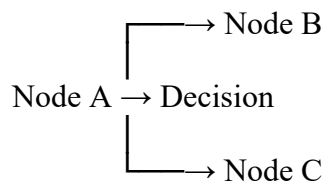
we can have:



28. What are Conditional Edges?

A **conditional edge** allows the graph to choose the next node based on a routing decision.

Example:



The decision might depend on:

- Current state
- User input
- Model output
- Validation result
- Database result
- Business rules

Interview definition

A conditional edge dynamically determines the next node based on the result of a routing function.

29. add_conditional_edges()

LangGraph provides:

`add_conditional_edges()`

to define dynamic routing.

Conceptually:

`builder.add_conditional_edges(`

```
"node_a",  
routing_function  
)
```

The routing function decides where execution should go.

30. Routing Function

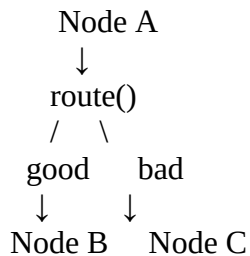
A routing function examines state and returns a routing decision.

Example:

```
def route(state):  
    if state["score"] >= 0.8:  
        return "good"  
    else:  
        return "bad"
```

Then the graph maps those decisions to nodes.

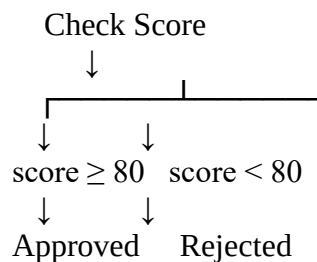
Conceptually:



31. Dynamic Routing

Dynamic routing means the execution path is determined **at runtime**.

Example:

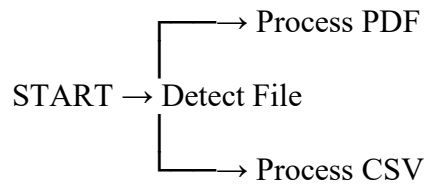


The graph doesn't always follow the same path.

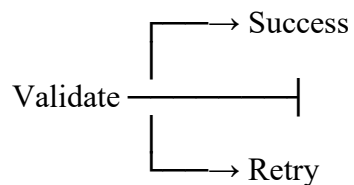
32. Branching

Branching occurs when a workflow can follow different paths.

Example:



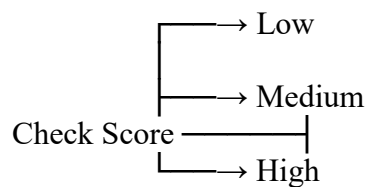
Another example:



33. Multiple Conditions

You can have more than two possible routes.

Example:



Routing function:

```
def route(state):  
    score = state["score"]  
  
    if score < 40:  
        return "low"  
    elif score < 80:  
        return "medium"
```

```
else:  
    return "high"
```

Conceptually:

score < 40



Low

40–79



Medium

80+



High

34. Route Based on State

This is one of the most common patterns.

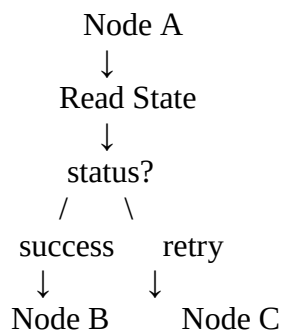
Suppose:

```
class State(TypedDict):  
    status: str
```

Routing:

```
def route(state):  
    if state["status"] == "success":  
        return "success"  
  
    return "retry"
```

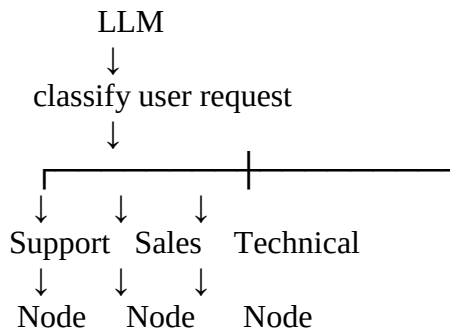
Workflow:



35. Route Based on Model Output

In LLM applications, routing can depend on model output.

Example:



The model might classify:

"How do I reset my password?"

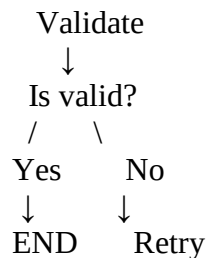
↓
SUPPORT

Then the router sends execution to the support node.

36. Route to END

A conditional route can terminate execution.

Example:



Conceptually:

```
def route(state):  
    if state["valid"]:
```

```
return "end"
```

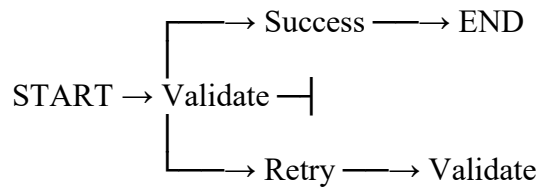
```
return "retry"
```

The routing map can associate "end" with END.

This is useful when the workflow should stop based on runtime conditions.

37. Complete Conditional Workflow Example

Consider:



This contains:

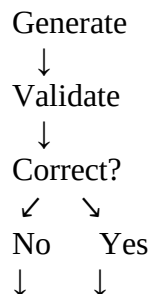
- Normal edges
- Conditional routing
- Branching
- A cycle
- Termination

This is a classic LangGraph workflow pattern.

38. Looping

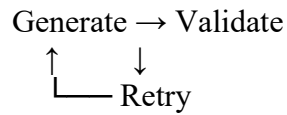
A loop means that execution returns to a previous node.

Example:



Generate END

This can be represented as:

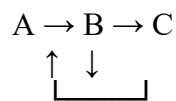


Loops are useful when you want to repeatedly perform an operation until a condition is satisfied.

39. Cycles

A **cycle** occurs when graph execution can return to an earlier node.

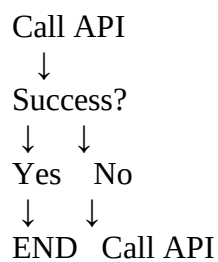
Example:



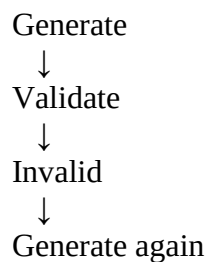
In LangGraph, cycles are useful for iterative workflows.

Examples:

Retry



Validation



40. Recursion / Iteration

In LangGraph discussions, you'll often encounter the concept of a **recursion limit**.

For graph execution, repeated cycles need a mechanism that prevents runaway execution.

Think:

$A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$

Without a stopping condition, the workflow could continue indefinitely.

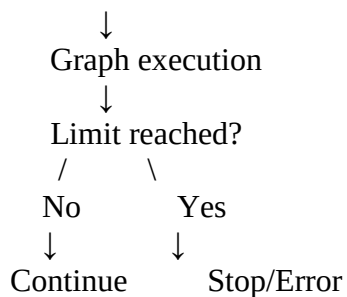
A recursion/step limit provides a safety boundary.

41. Maximum Recursion Limit

LangGraph provides a configurable recursion/step limit for graph execution.

The exact configuration depends on the LangGraph API/version, but conceptually:

Maximum allowed execution steps



For example, conceptually:

```
graph.invoke(  
    input_state,  
    config={  
        "recursion_limit": 10  
    }  
)
```

The important interview concept is not memorizing the syntax but understanding **why the limit exists**:

It prevents a cyclic workflow from executing indefinitely.

42. Preventing Infinite Loops

Never create a cycle without a termination strategy.

Bad design:

```
A
↓
B
↓
A
↓
B
↓
A
↓
...
```

There is no exit condition.

Better:

```
A
↓
B
↓
Check
↙  ↘
Retry Success
↓    ↓
A    END
```

You should have:

1. Exit condition

```
if success:
    END
```

2. Maximum attempts

```
attempt < 3
```

3. Recursion/step limit

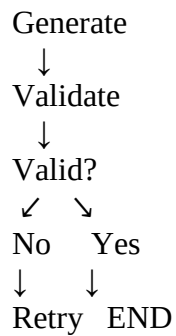
Use the graph execution limit as a safety mechanism.

43. Workflow Termination

Every cyclic workflow should eventually reach:

END

For example:

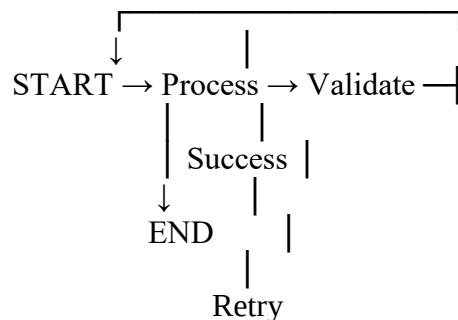


You can terminate based on:

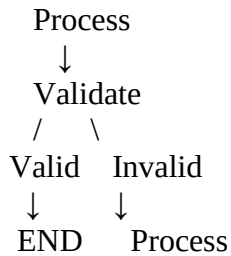
- Success
 - Failure
 - Maximum attempts
 - Validation result
 - User decision
 - Business condition
-

44. Retry Pattern

One of the most useful LangGraph patterns is retry.



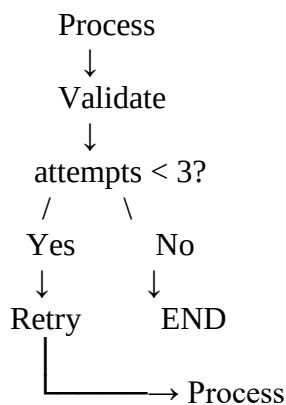
More clearly:



This is much easier to implement with a graph than with deeply nested procedural logic.

45. Maximum Attempt Pattern

A safer retry workflow:



Now there are two termination conditions:

Success → END

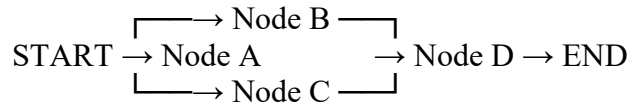
or:

Maximum attempts → END

This is a good production design.

46. Conditional Routing Architecture

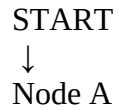
Your example:



Let's understand it step-by-step.

Step 1

Execution begins:

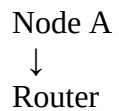


Step 2

Node A produces/updates state.

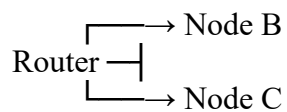
Step 3

A routing function evaluates state:



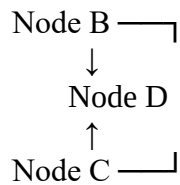
Step 4

Router chooses:



Step 5

Both paths can eventually converge:

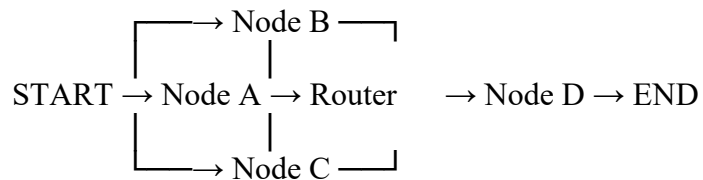


Step 6

Node D finishes:

Node D
↓
END

Complete:



47. Conditional Edge vs Normal Edge

Very important interview distinction.

Normal edge

```
builder.add_edge("A", "B")
```

Means:

$A \rightarrow B$

The next destination is fixed.

Conditional edge

```
builder.add_conditional_edges(  
    "A",  
    router  
)
```

Means:

A
↓
Router
↓
Choose destination

The destination depends on runtime logic.

Easy memory trick

add_edge() = fixed path

add_conditional_edges() = dynamic path

48. Nodes vs Conditional Routing

A good design separates:

Node

Does the actual work.

```
def validate(state):
```

```
...
```

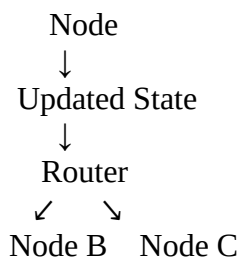
Router

Decides where to go.

```
def route(state):
```

```
...
```

Conceptually:



This separation makes workflows easier to understand and maintain.

49. Complete Module 4 Example

A simple validation/retry workflow:

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

```
class State(TypedDict):
    attempts: int
    success: bool

def process(state):
    attempts = state["attempts"] + 1

    return {
        "attempts": attempts
    }

def validate(state):
    return {
        "success": state["attempts"] >= 3
    }

def route(state):
    if state["success"]:
        return "done"

    if state["attempts"] >= 5:
        return "done"

    return "retry"

builder = StateGraph(State)

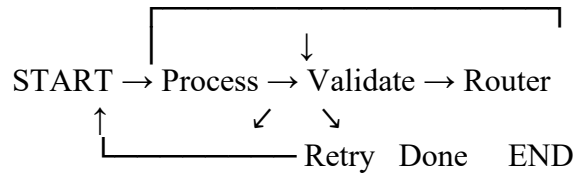
builder.add_node("process", process)
builder.add_node("validate", validate)

builder.add_edge(START, "process")
builder.add_edge("process", "validate")

builder.add_conditional_edges(
    "validate",
    route,
    {
        "retry": "process",
        "done": END
    }
)
```

```
graph = builder.compile()
```

Architecture:



The exact business logic here is just for learning the pattern.

Module 5 — Graph Compilation & Execution

Goal

Understand what happens between:

Graph Definition



Compilation



Execution



Result

The key idea is:

StateGraph is where you define the workflow. **compile()** turns that definition into an executable graph. **invoke()** or **ainvoke()** runs it.

1. Building a Graph

A LangGraph application generally follows these steps:

Define State



Create StateGraph



Add Nodes



Add Edges



Compile



Invoke



Result

For example:

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

```
class State(TypedDict):
    message: str
```

```
def node_a(state):
    return {
        "message": "Hello"
    }
```

```
def node_b(state):
    return {
        "message": state["message"] + " World"
    }
```

```
builder = StateGraph(State)
```

```
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
```

```
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", END)
```

At this point, you have **defined the graph**, but you haven't executed it yet.

2. Adding Nodes

Nodes represent operations.

```
builder.add_node("node_a", node_a)
```

```
builder.add_node("node_b", node_b)
```

Conceptually:

```
State
↓
Node A
↓
Node B
```

The first argument is the node's name:

```
"node_a"
```

The second argument is the function:

```
node_a
```

3. Adding Edges

Edges define the execution flow.

```
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
builder.add_edge("node_b", END)
```

Graph:

```
START
↓
Node A
↓
Node B
↓
END
```

Until the nodes and edges are connected appropriately, the graph isn't a complete executable workflow.

4. What is compile()?

compile() converts the graph definition into a **compiled executable graph**.

Example:

```
graph = builder.compile()
```

Before:

StateGraph Builder



Nodes + Edges



Not yet executable

After:

```
builder.compile()
```



Compiled Graph



Can be invoked

Simple definition

compile() takes the graph you've constructed and prepares it for execution, including validating the graph structure and producing a runnable graph object.

5. Why Do We Compile a LangGraph?

This is a common interview question.

Compilation provides a boundary between:

Graph construction

Define State

Add Nodes

Add Edges

and:

Graph execution

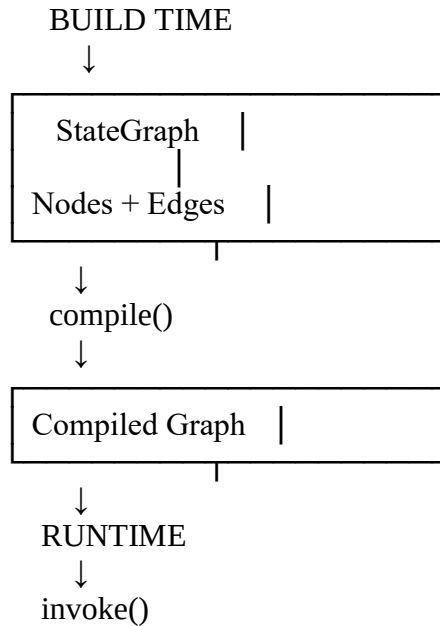
Invoke



Execute Nodes

↓
Update State
↓
Return Result

Conceptually:



Interview answer

We compile a LangGraph to transform the graph definition into a validated executable graph that can be invoked at runtime.

6. StateGraph vs Compiled Graph

This distinction is extremely important.

StateGraph	Compiled Graph
Used to construct the workflow	Used to execute the workflow
Add nodes	Invoke
Add edges	Stream
Define graph structure	Run graph
Builder	Runnable
Before compilation	After compilation

Think:

```
StateGraph
↓
BUILD
↓
compile()
↓
Compiled Graph
↓
RUN
```

Easy interview answer

StateGraph is the builder used to define the graph, while the compiled graph is the executable representation produced by compile().

7. compile()

Basic usage:

```
graph = builder.compile()
```

After compilation:

```
result = graph.invoke(input_state)
```

So:

```
builder
↓
compile()
↓
graph
↓
invoke()
```

8. What is invoke()?

invoke() executes the graph **synchronously**.

Example:

```
result = graph.invoke({  
    "message": ""  
})
```

Flow:

```
Input State  
↓  
invoke()  
↓  
START  
↓  
Node A  
↓  
Node B  
↓  
END  
↓  
Output State
```

The result is the final state returned by the graph.

9. Input State

The input state is the initial data provided to the graph.

Suppose:

```
class State(TypedDict):  
    name: str  
    message: str
```

You can invoke:

```
result = graph.invoke({  
    "name": "Prathamesh",  
    "message": ""  
})
```

This becomes the starting state of the workflow.

Conceptually:

Input

```
{  
  name: "Prathamesh",  
  message: ""  
}
```

↓
Graph

10. Output State

After all nodes execute, the graph returns the resulting state.

Example:

```
result = graph.invoke({  
  "name": "Prathamesh",  
  "message": ""  
})
```

Result might conceptually be:

```
{  
  "name": "Prathamesh",  
  "message": "Hello Prathamesh"  
}
```

So:

```
Input State  
↓  
Graph Execution  
↓  
Output State
```

11. invoke() Mental Model

Remember:

```
invoke()  
↓  
Give graph input  
↓  
Graph executes  
↓  
Nodes process state
```

↓
Edges determine next steps
↓
END
↓
Return final state

12. Async Execution

For asynchronous workflows, LangGraph provides asynchronous execution methods.

The most important one here is:

`ainvoke()`

Example:

```
result = await graph.ainvoke({  
    "message": "Hello"  
})
```

Notice:

`await`

because the operation is asynchronous.

13. `invoke()` vs `ainvoke()`

<code>invoke()</code>	<code>ainvoke()</code>
Synchronous	Asynchronous
Normal Python call	Uses <code>await</code>
<code>graph.invoke(...)</code>	<code>await graph.ainvoke(...)</code>
Blocks while executing	Designed for async execution
Useful for sync workflows	Useful for async workflows

Example

Synchronous:

```
result = graph.invoke(input_state)
```

Asynchronous:

```
result = await graph.ainvoke(input_state)
```

Interview answer

invoke() executes the graph synchronously, while ainvoke() provides asynchronous execution and is used with await.

14. Async Node + Async Graph

Suppose a node is asynchronous:

```
async def call_api(state):  
    result = await some_api_call()  
  
    return {  
        "result": result  
    }
```

You can execute the graph asynchronously:

```
result = await graph.ainvoke(input_state)
```

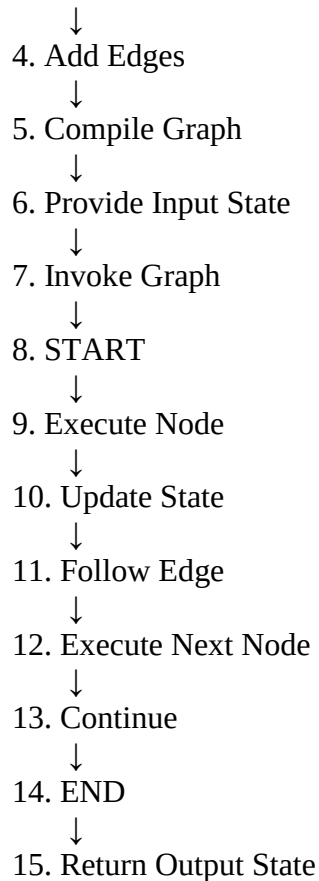
Conceptually:

```
Async Graph  
  ↓  
Async Node  
  ↓  
await API  
  ↓  
Result
```

15. Execution Lifecycle

A complete graph lifecycle can be visualized as:

```
1. Define State  
  ↓  
2. Create StateGraph  
  ↓  
3. Add Nodes
```

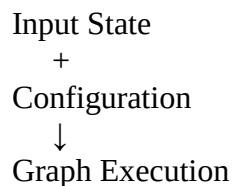


This is one of the most important diagrams to remember.

16. Graph Configuration

Graph execution can use configuration information in addition to the input state.

Conceptually:



Configuration can be used for things such as:

- Runtime execution settings
- Recursion limits
- Metadata

- Tags
- Checkpoint/thread information
- Other runtime controls depending on the application

Example conceptually:

```
config = {  
  "recursion_limit": 10  
}
```

```
result = graph.invoke(  
  input_state,  
  config=config  
)
```

The exact configuration options depend on the LangGraph feature being used.

17. Runtime Configuration

Runtime configuration means:

Settings supplied when the graph is executed rather than hard-coded into the graph definition.

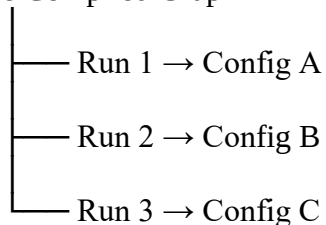
For example:

```
result = graph.invoke(  
  input_state,  
  config={  
    "recursion_limit": 10  
  }  
)
```

This allows different executions to use different settings.

Conceptually:

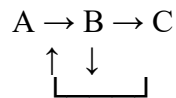
Same Compiled Graph



18. Recursion Limits

Recursion limits are especially important for **cyclic graphs**.

Suppose:



If there is no termination condition:

$A \rightarrow B \rightarrow C \rightarrow B \rightarrow C \rightarrow B \rightarrow C \rightarrow \dots$

The workflow could continue indefinitely.

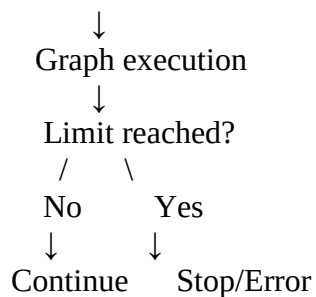
A recursion/step limit provides a safety mechanism.

Example conceptually:

```
graph.invoke(  
  input_state,  
  config={  
    "recursion_limit": 10  
  }  
)
```

Think:

Maximum allowed execution depth/steps



Important

A recursion limit **does not replace proper loop termination logic**.

You should still design your workflow with an explicit exit condition.

19. Execution Errors

Errors can happen during graph execution.

For example:

Node A
↓
Node B
↓
ERROR

Possible causes include:

- Python exception
- Invalid input
- API failure
- LLM failure
- Database failure
- Invalid state update
- Routing error
- Recursion limit reached

Conceptually:

Graph
↓
Node
↓
Exception
↓
Execution fails

Good applications should handle expected errors appropriately.

20. Graph Validation

Compilation is also an important validation stage.

The graph should have a valid structure before execution.

For example, you should ensure:

START



Reachable Nodes



END

Problems can include:

- Incorrect node references
- Invalid edges
- Missing required graph connections
- Invalid routing destinations
- Other graph construction problems

The important idea:

Compilation provides a validation boundary before runtime execution.

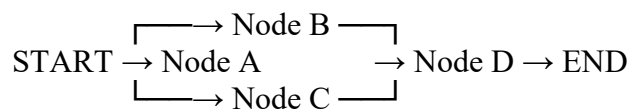
21. Graph Visualization

Graph visualization helps you understand the workflow structure.

Instead of reading:

```
builder.add_node(...)
builder.add_edge(...)
builder.add_conditional_edges(...)
```

you can visualize:



This becomes very useful when your graph contains:

- Many nodes
- Branches
- Loops
- Multiple paths
- Complex routing

Why visualization matters

It helps you:

- Debug workflow design
- Understand execution paths
- Explain architecture
- Find missing connections
- Understand complex graphs

22. Building vs Compiling vs Executing

This distinction is worth memorizing.

Building

StateGraph
↓
Add nodes
↓
Add edges

Compiling

compile()
↓
Executable Graph

Executing

invoke()
↓
Run Graph

Complete:

BUILD
↓
COMPILE
↓
EXECUTE

23. Complete Example

```
from typing_extensions import TypedDict
```

Prathamesh Arvind Jadhav

```
from langgraph.graph import StateGraph, START, END
```

```
class State(TypedDict):  
    message: str
```

```
def greet(state):  
    return {  
        "message": "Hello"  
    }
```

```
def personalize(state):  
    return {  
        "message": state["message"] + " Prathamesh"  
    }
```

```
# 1. Create builder
```

```
builder = StateGraph(State)
```

```
# 2. Add nodes
```

```
builder.add_node("greet", greet)
```

```
builder.add_node("personalize", personalize)
```

```
# 3. Add edges
```

```
builder.add_edge(START, "greet")
```

```
builder.add_edge("greet", "personalize")
```

```
builder.add_edge("personalize", END)
```

```
# 4. Compile
```

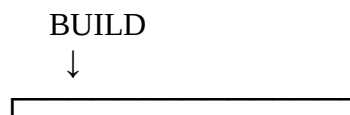
```
graph = builder.compile()
```

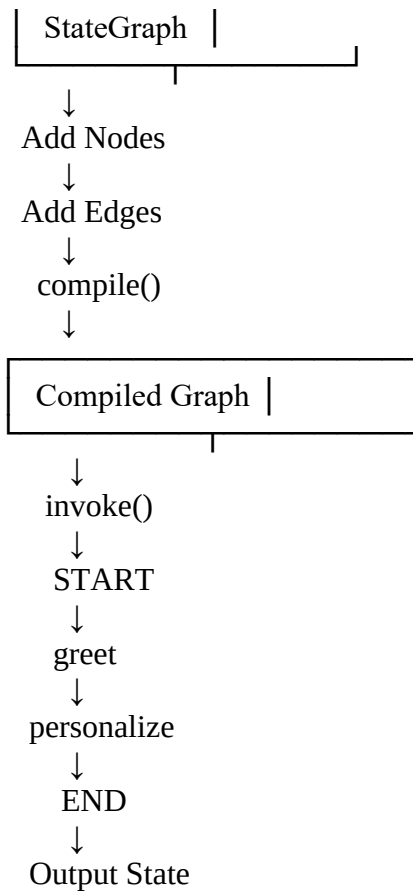
```
# 5. Execute
```

```
result = graph.invoke({  
    "message": ""  
})
```

```
print(result)
```

Architecture:





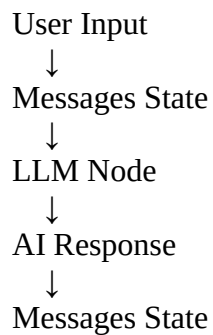
Module 6 — Messages, LLMs & LangChain Integration

Goal

Understand how LangGraph manages **conversation messages and LLM calls**.

We are **not covering Agentic AI here**.

The focus is:



24. What are Messages in LangGraph?

In LLM applications, information is often represented as a sequence of messages.

Example:

Human: What is LangGraph?

AI: LangGraph is a graph-based framework.

Human: Why do we use it?

AI: It is useful for stateful workflows.

These messages form the conversation history.

25. Types of Messages

The common message roles you should know are:

System

Human

AI

Tool

They represent different sources/purposes of messages.

26. Human Messages

A **Human message** represents input from the user.

Example:

Human:

"What is LangGraph?"

Conceptually:

```
HumanMessage(  
    content="What is LangGraph?"  
)
```

Flow:

User
↓
Human Message
↓
Graph

27. AI Messages

An **AI message** represents a response generated by an AI/LLM.

Example:

AI:
"LangGraph is a framework for building stateful workflows."

Conceptually:

```
AIMessage(  
    content="LangGraph is a framework..."  
)
```

Flow:

LLM
↓
AI Message
↓
Messages State

28. System Messages

A **System message** provides instructions or context to the model.

Example:

System:
"You are a helpful Python tutor."

Then:

System
↓

Human
↓
LLM
↓
AI

The system message generally defines the model's behavior/context.

29. Tool Messages

A **Tool message** represents the result of a tool interaction.

For example, conceptually:

Model requests operation
↓
Tool executes
↓
Tool result
↓
Tool Message

Example:

Tool:
"Weather in Pune: 28°C"

For the basic LangGraph concepts, remember:

Tool messages represent information returned from tool execution.

Detailed tool-calling/agent patterns can be studied separately.

30. Message History

Message history is the collection of previous messages.

Example:

[
 Human: "What is Python?",
 AI: "Python is a programming language.",

```
Human: "What is a list?",  
AI: "A list is a collection of values."  
]
```

This history provides conversational context to the LLM.

31. Why Message History Matters

Consider:

User:
What is Python?

AI:
Python is a programming language.

User:
What are lists?

The LLM needs previous context to understand the conversation.

Message history allows the application to maintain:

```
Previous conversation  
  ↓  
Current message  
  ↓  
LLM  
  ↓  
Context-aware response
```

32. MessagesState

MessagesState is a convenient prebuilt state schema for workflows that operate around messages.

Conceptually:

```
MessagesState  
├── messages  
│   └── HumanMessage
```



Instead of manually designing all message-state behavior, LangGraph provides this reusable structure.

33. Using MessagesState

A simple pattern is:

```
from langgraph.graph import MessagesState, StateGraph
```

Then:

```
builder = StateGraph(MessagesState)
```

Now the graph's state is designed around messages.

Conceptually:

```
MessagesState
  ↓
messages
  ↓
LLM Node
  ↓
new AI message
```

34. Adding Messages

A user message can be provided as graph input.

Conceptually:

```
from langchain_core.messages import HumanMessage
```

```
input_state = {
    "messages": [
        HumanMessage(content="What is LangGraph?")
    ]
}
```

Then the graph can process it.

35. Updating Messages

An LLM node can produce an AI message.

Conceptually:

```
def call_model(state):  
    response = model.invoke(state["messages"])  
  
    return {  
        "messages": [response]  
    }
```

Flow:

Existing Messages



LLM Node



AI Message



Updated Messages

The important point is that message state needs appropriate update/reducer behavior so that conversation history is maintained correctly.

36. Message Reducers

Message state often needs **accumulation**, rather than simply replacing the existing messages.

Suppose:

Existing:

Human → "Hello"

Node produces:

AI → "Hi!"

We want:

Human → "Hello"

AI → "Hi!"

not:

AI → "Hi!"

This is why message state uses appropriate reducer/message-management behavior.

Conceptually:

Existing Messages

+

New Messages

↓

Updated Message History

37. Message Flow

The fundamental message workflow is:

User Input

↓

HumanMessage

↓

MessagesState

↓

LLM Node

↓

AIMessage

↓

MessagesState

↓

Next Node / END

For multiple turns:

Human

↓

AI

↓

Human

↓

AI

↓

Human



AI

The state maintains the conversation context.

38. LLM Node

An **LLM node** is a LangGraph node that calls a language model.

Example conceptually:

```
def call_model(state):
    response = model.invoke(state["messages"])

    return {
        "messages": [response]
    }
```

The node:

1. Reads messages from state.
2. Sends them to the model.
3. Receives model response.
4. Returns the response as a state update.

Architecture:

MessagesState



LLM Node



Chat Model



AI Message



MessagesState

39. Chat Model Integration

LangGraph can work with chat models through the model interfaces provided by the surrounding LLM ecosystem.

Conceptually:

LangGraph Node



Chat Model



Model Response



AI Message

The node becomes the bridge between:

Graph State

and:

LLM

40. LangChain Model Integration

One of LangGraph's strengths is that you can use LangChain model components inside nodes.

For example:

LangGraph



LLM Node



LangChain Chat Model



Response

This demonstrates the relationship:

LangChain



Provides model/component abstractions

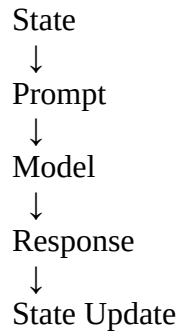
LangGraph



Orchestrates them in a workflow

41. Prompt + Model + State

A typical LLM node can combine:



Example conceptually:

```
def generate_answer(state):
    question = state["question"]

    prompt = f"""
    Answer the following question:

    {question}
    """

    response = model.invoke(prompt)

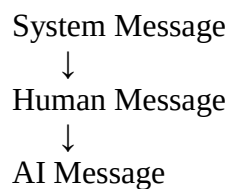
    return {
        "answer": response.content
    }
```

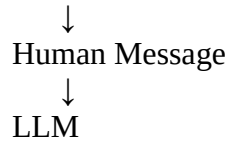
This is a basic LLM-powered LangGraph node.

42. Using Messages Instead of a Simple Prompt

For conversational applications, you usually work with message objects.

Conceptually:





The LLM receives the conversation context as messages.

43. Structured Output

LLMs normally return natural-language output.

For example:

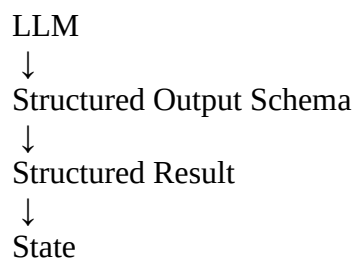
"The answer is LangGraph."

But applications often need structured data:

```
{  
  "topic": "LangGraph",  
  "confidence": 0.95  
}
```

Structured output allows the model response to follow a predefined structure.

Conceptually:



This is useful when the next graph node needs predictable fields.

44. Why Structured Output is Useful in LangGraph

Suppose a routing node needs:

category
confidence

Instead of parsing:

"I think this is probably a technical question..."

you can work with structured data:

```
{  
  "category": "technical",  
  "confidence": 0.95  
}
```

Then routing can use:

```
category == "technical"
```

↓

Technical Node

This makes workflows more reliable.

45. Model Response Handling

An LLM response generally needs to be converted into something useful for your graph.

Typical flow:

```
State  
↓  
LLM  
↓  
Model Response  
↓  
Extract Content / Structured Data  
↓  
State Update
```

For example:

```
def call_model(state):  
    response = model.invoke(state["messages"])  
  
    return {  
        "messages": [response]  
    }
```

Or:

```
def call_model(state):  
    response = model.invoke(state["question"])  
  
    return {  
        "answer": response.content  
    }
```

46. Streaming Model Responses

Instead of waiting for the complete result:

"LangGraph is a framework..."

streaming allows output to arrive progressively:

Lang
LangGraph
LangGraph is
LangGraph is a
...

Conceptually:

```
User  
↓  
Graph  
↓  
LLM  
↓  
Token/Chunk  
↓  
Token/Chunk  
↓  
Token/Chunk  
↓  
Final Response
```

LangGraph supports streaming execution patterns, allowing applications to observe graph/model output as execution progresses.

47. Why Streaming is Useful

Streaming improves user experience for applications where model generation takes noticeable time.

Instead of:

User waits...

User waits...

User waits...

Complete answer appears

you can have:

User



Generating...



"LangGraph"



"LangGraph is"



"LangGraph is a framework..."

This is especially useful for chat applications.

48. Multiple LLM Nodes

A graph can contain multiple LLM nodes.

Example:

START



LLM Node 1



LLM Node 2



LLM Node 3

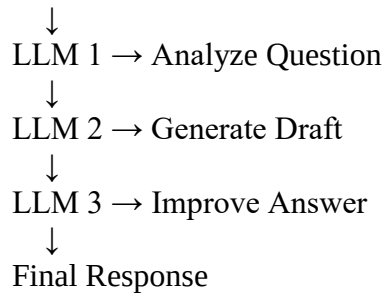


END

Each node can have a different purpose.

For example:

User Input



This is still a normal workflow; it does not require agentic behavior.

49. Conversation Context Management

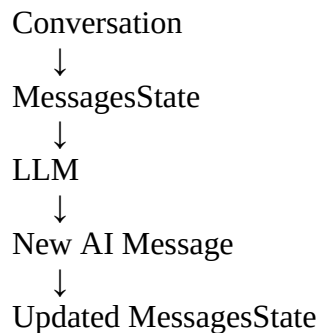
The LLM needs the right context.

For example:

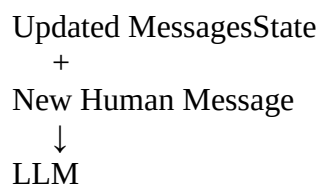
Message 1 → Human
Message 2 → AI
Message 3 → Human
Message 4 → AI

The state maintains these messages.

Flow:



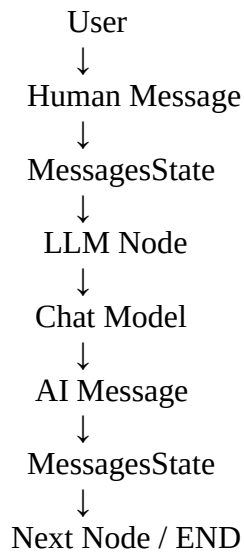
On the next turn:



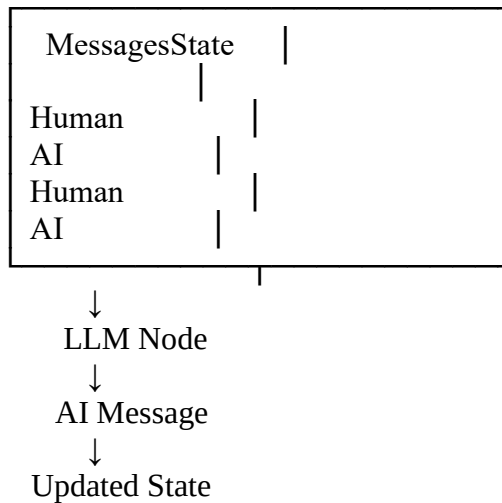
This is how conversational context can be maintained within a graph workflow.

50. Complete LLM + LangGraph Architecture

The architecture you should remember is:



For multiple turns:



Module 7 — Persistence, Checkpointing & Memory

Goal

Understand how LangGraph can **save graph state, maintain state across runs, resume interrupted workflows, and inspect previous execution states.**

This is one of the most important LangGraph topics for interviews because it explains how a workflow can behave like a **long-running application** rather than a one-time function call.

1. What is Persistence?

Persistence means saving the state of a LangGraph workflow so that it can be retrieved later.

Without persistence:

```
graph TD; Input --> Executes[Graph executes]; Executes --> Final[Final state]; Final --> Ends[Process ends];
```

Once the execution ends, the in-memory state may no longer be available.

With persistence:

```
graph TD; Input --> Executes[Graph executes]; Executes --> State[State]; State --> Checkpoint[Save checkpoint]; Checkpoint --> Storage[Storage];
```

Later:

```
graph TD; Storage --> Restore[Restore state]; Restore --> Continue[Continue workflow];
```

Simple definition

Persistence is the ability to save graph state so that it can be accessed, restored, or continued across executions.

2. Why is Persistence Required?

Persistence is useful when your application needs to remember workflow state.

Common use cases:

- Chat conversations
- Long-running workflows
- Human approval
- Workflow recovery
- Background jobs
- Multi-step processing
- Debugging previous states
- Resuming after failure
- Maintaining state between requests

For example:

User
↓
Start application
↓
Conversation
↓
Application restarted
↓
Load previous state
↓
Continue conversation

Without persistence, the previous workflow state may be lost.

3. What is Checkpointing?

Checkpointing means saving the graph's state at specific points during execution.

Think of it like saving a game:

Game

Prathamesh Arvind Jadhav

↓
Reach Level 5
↓
SAVE
↓
Continue
↓
Game crashes
↓
LOAD
↓
Continue from saved point

LangGraph works similarly:

Node A
↓
State
↓
Checkpoint
↓
Node B
↓
State
↓
Checkpoint

Simple definition

A checkpoint is a persisted snapshot of graph state at a particular point in workflow execution.

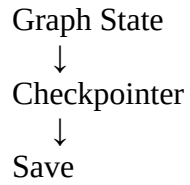
4. What is a Checkpointer?

A **checkpointer** is the component responsible for saving and retrieving checkpoints.

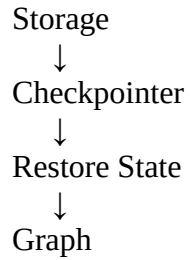
Conceptually:

Graph
↓
Checkpointer
↓
Storage

During execution:



During recovery:



So:

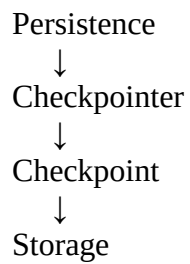
Checkpoint = mechanism that persists graph execution state.

5. Checkpointer vs Checkpoint

Don't confuse these terms.

Term	Meaning
Checkpoint	Saved snapshot of graph state
Checkpointer	Component that saves/loads checkpoints
Persistence	Overall ability to retain state
Storage	Where persisted data is stored

Think:



6. Thread ID

A **thread** identifies a particular workflow execution/conversation.

A thread ID allows LangGraph to distinguish one conversation or workflow from another.

Example:

```
thread_id = "user_123"
```

Another user:

```
thread_id = "user_456"
```

The states remain separate:

Thread A
↓
State A
↓
Checkpoints A

Thread B
↓
State B
↓
Checkpoints B

Important

The thread ID is essentially an identifier used to associate checkpoints with a particular execution thread.

7. Why is Thread ID Important?

Imagine a chatbot with 1,000 users.

You don't want:

User A conversation
↓
User B conversation
↓

User C conversation

to become one shared state.

Instead:

Thread 101 → User A conversation

Thread 102 → User B conversation

Thread 103 → User C conversation

Each thread has its own state/checkpoint history.

8. Thread-Based State

Think of thread-based state as:

Thread ID
↓
Checkpoint history
↓
Current state

Example:

thread_id = "conversation_001"

might contain:

Checkpoint 1
↓
Checkpoint 2
↓
Checkpoint 3
↓
Current State

Another thread:

thread_id = "conversation_002"

has its own history.

9. State Snapshots

A **state snapshot** represents the state of the graph at a particular point in execution.

For example:

After Node A:

```
{  
  "name": "Prathamesh",  
  "status": "processing"  
}
```

Then:

After Node B:

```
{  
  "name": "Prathamesh",  
  "status": "completed"  
}
```

Conceptually:

Checkpoint 1
↓
State Snapshot 1

Checkpoint 2
↓
State Snapshot 2

Checkpoint 3
↓
State Snapshot 3

Snapshots are useful for:

- Debugging
- Recovery
- History inspection
- Resuming
- Understanding execution

10. Short-Term Memory

In conversational applications, **short-term memory** refers to information available within the ongoing conversation/thread.

Example:

User:

My name is Prathamesh.

AI:

Nice to meet you.

User:

What is my name?

The application can use the current conversation state:

Messages

↓

Human: My name is Prathamesh

AI: Nice to meet you

Human: What is my name?

This information is associated with the current thread.

Conceptually:

Thread

↓

Current conversation state

↓

Short-term memory

11. Long-Term Memory

Long-term memory refers to information that needs to persist beyond a single conversation/thread or be reused across sessions.

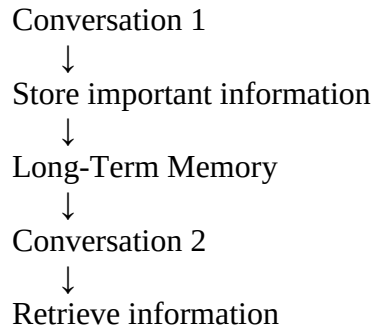
For example:

User preference:

"Prefers Python examples."

That information might be useful in another conversation.

Conceptually:



Important distinction

Don't simply think:

Checkpoint = Long-term memory

They are related but different concepts.

Checkpointing primarily preserves workflow execution state.

Memory refers to information an application intentionally retains and retrieves for future use.

12. State vs Memory

This is an important interview question.

State	Memory
Current workflow data	Information retained for future use
Used during execution	Can persist beyond an execution
Represents current graph context	Represents remembered information
Can be checkpointed	Can be stored/retrieved separately
Often thread-specific	May be thread/user/application-level

Example:

```
STATE
{
  "current_step": "approval",
  "amount": 5000
}
```

Memory could contain:

```
USER PREFERENCE
{
  "preferred_language": "Python"
}
```

Interview answer

State represents the data needed by the workflow at a particular point in execution, while memory refers to information intentionally retained so it can be reused later. Checkpointing can persist workflow state, but memory and checkpointing are not the same concept.

13. In-Memory Checkpointing

For development and experimentation, you can use an in-memory checkpointer.

Conceptually:

```
Graph
↓
In-Memory Checkpointer
↓
RAM
```

Advantages:

- Easy to use
- Fast
- Good for testing
- No external database required

Disadvantage:

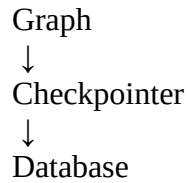
```
Application stops
↓
RAM cleared
↓
Checkpoint lost
```

So it is generally not suitable when you need durable persistence across application restarts.

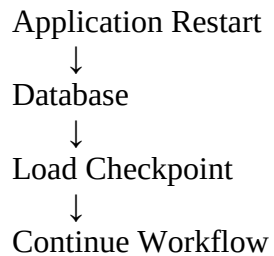
14. Database Checkpointing

For production applications, checkpoints can be stored in persistent storage.

Conceptually:



Then:



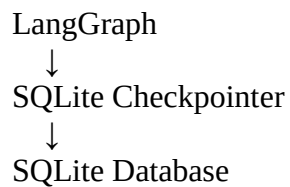
Advantages:

- Durable
- Survives application restart
- Suitable for production
- Supports recovery

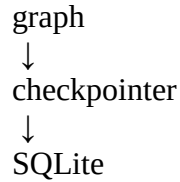
15. SQLite Checkpointer

SQLite can be used for checkpoint persistence, particularly for local development and smaller applications.

Architecture:



Example conceptually:



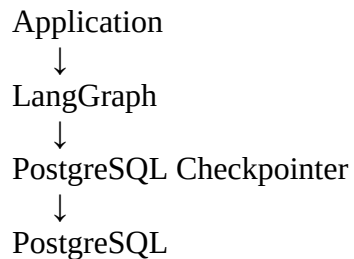
Useful for:

- Learning
- Local development
- Prototypes
- Small applications

16. PostgreSQL Checkpointer

PostgreSQL is more suitable when your application already uses PostgreSQL and requires durable, production-oriented persistence.

Architecture:

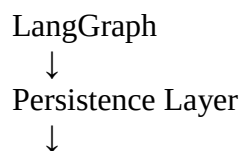


Multiple application instances can work against shared persistent storage when the infrastructure and configuration are designed appropriately.

17. Redis-Based Persistence Concepts

Redis can also be used in persistence architectures depending on the LangGraph/LangChain integration and the persistence requirements.

Conceptually:



Redis

Redis is commonly used when applications need:

- Fast access
- Distributed application state
- Caching
- Temporary/session-oriented data

But remember:

Redis persistence and durable workflow checkpointing are architectural choices; choose the storage/checkpointer supported by your LangGraph version and deployment requirements.

18. Resuming Execution

One of the biggest benefits of checkpointing is the ability to resume a workflow.

Imagine:

```
START
↓
Node A
↓
Node B
↓
Node C
```

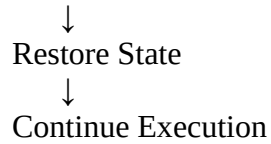
Suppose execution reaches Node B and the application stops.

If the state was checkpointed:

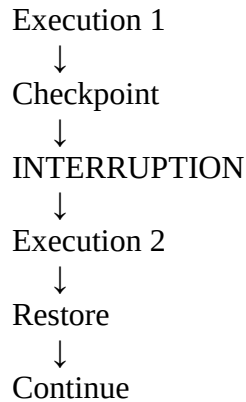
```
Node A
↓
Checkpoint
↓
Node B
↓
Application stops
```

Later:

Load Checkpoint



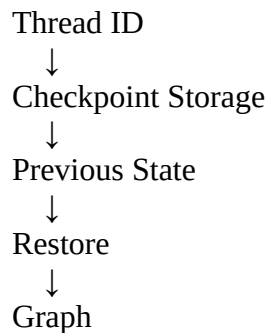
Conceptually:



19. Recovering State

Recovery means retrieving a previously persisted state.

Conceptually:



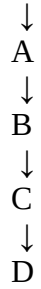
This is particularly useful for long-running workflows.

20. Time-Travel Debugging

One of LangGraph's powerful concepts is the ability to inspect previous execution states and, depending on the workflow design and APIs being used, replay or branch execution from historical states.

Imagine:

START



Snapshots:

S1 → after A

S2 → after B

S3 → after C

S4 → after D

You can inspect:

S1

S2

S3

S4

This makes debugging easier because you can ask:

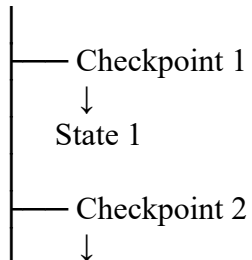
"What was the state when the workflow reached Node C?"

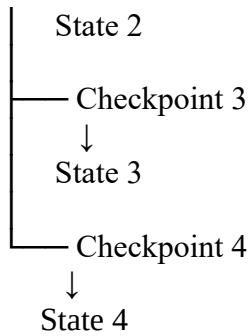
21. State History

A persisted workflow can have a history of states/checkpoints.

Conceptually:

Thread



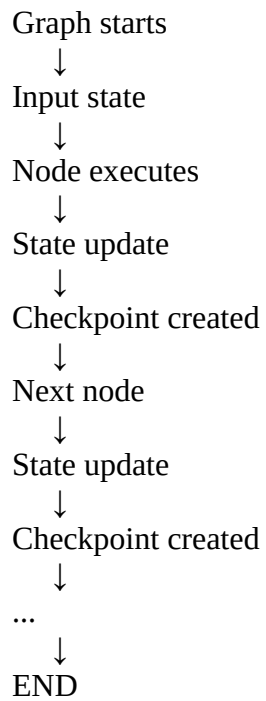


This history helps with:

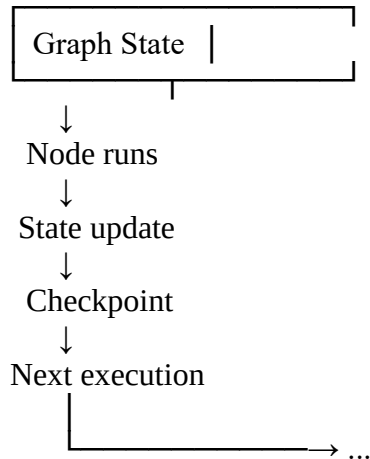
- Debugging
- Auditing
- Recovery
- Understanding workflow execution
- Replaying/branching scenarios

22. Checkpoint Lifecycle

A simplified lifecycle:



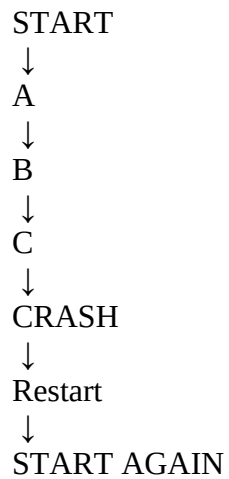
Conceptually:



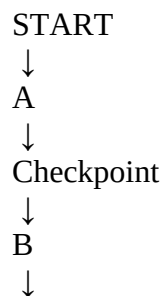
23. Durable Execution

Durable execution means designing workflows so that progress can survive failures and execution can continue from persisted state rather than starting from the beginning.

Without durability:



With durable execution:



Checkpoint



CRASH



Restart



Restore



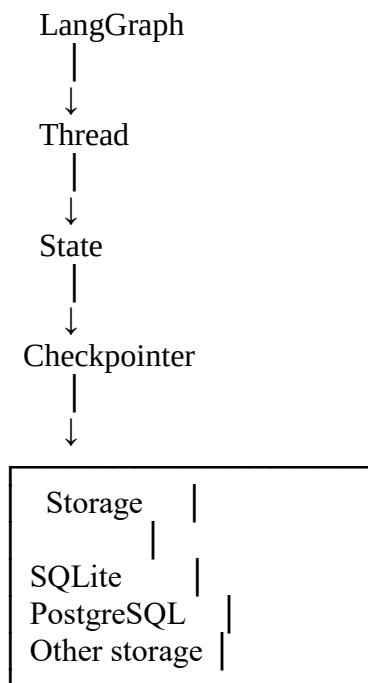
Continue

Simple definition

Durable execution allows long-running workflows to persist progress and recover from interruptions or failures.

24. Complete Persistence Architecture

Remember this architecture:

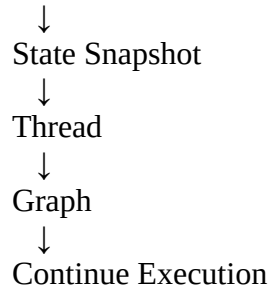


When the workflow needs recovery:

Storage

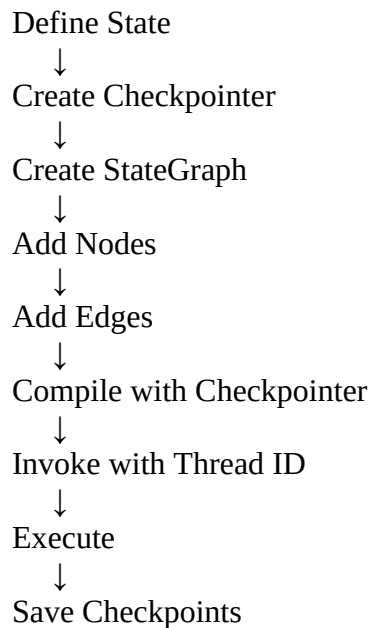


Checkpoint



25. Complete Example Concept

A persistent graph generally follows this pattern:



Conceptually:

checkpointer = ...

```
graph = builder.compile(  
    checkpointer=checkpointer  
)
```

Then execution is associated with a thread/configuration:

```
config = {  
    "configurable": {  
        "thread_id": "conversation-1"  
    }  
}
```

```
}
```

Then:

```
graph.invoke(  
    input_state,  
    config=config  
)
```

The exact checkpointer setup differs depending on the backend and current LangGraph version.

Module 8 — Human-in-the-Loop, Interrupts & Streaming

Goal

Learn how to make a workflow:

```
PAUSE  
↓  
WAIT FOR HUMAN  
↓  
GET DECISION  
↓  
RESUME
```

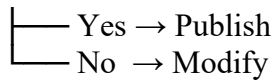
This is extremely useful for workflows where the application should **not make an important decision completely automatically**.

26. What is Human-in-the-Loop?

Human-in-the-Loop (HITL) means a human can inspect, approve, reject, modify, or provide information during workflow execution.

Example:

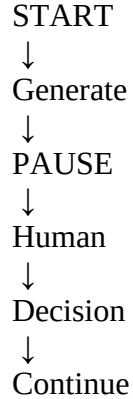
```
START  
↓  
Generate Report  
↓  
Human Approval  
↓  
Approved?
```



Instead of:

START → Generate → Publish

we have:

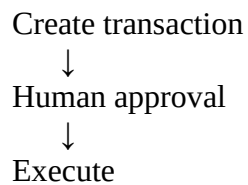


27. Why Human-in-the-Loop?

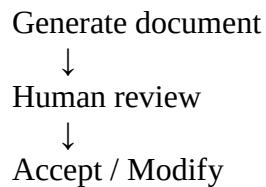
HITL is useful when human judgment is required.

Examples:

Financial approval



Document review



Sensitive operation

Prepare operation
↓
Human confirmation
↓
Execute

The general principle is:

Let the system automate routine processing while allowing humans to control important decisions.

28. Interrupts

An **interrupt** pauses graph execution at a particular point.

Conceptually:

Node A
↓
Node B
↓
INTERRUPT
↓
PAUSED

The workflow does not continue until it receives the information required to resume.

29. interrupt()

LangGraph provides the `interrupt()` mechanism for pausing execution and requesting external/human input.

Conceptually:

```
from langgraph.types import interrupt
```

Inside a node:

```
def approval_node(state):  
    decision = interrupt(  
        "Please approve this operation"  
    )
```

```
return {  
  "decision": decision  
}
```

Conceptually:

Node
↓
interrupt()
↓
PAUSE
↓
Human input
↓
Resume

30. Important: Interrupt Needs Persistence

This is a very important connection between Modules 7 and 8.

HITL often requires the workflow to remain paused across requests.

Therefore:

Interrupt
+
Checkpointing
↓
Persistent paused workflow

Without persistence, managing a paused workflow across application requests becomes much harder.

Interview point

Checkpointing is important for HITL because the paused workflow state needs to be persisted so the workflow can later be resumed.

31. Pausing Graph Execution

Consider:

Prathamesh Arvind Jadhav

```
START
↓
Process Request
↓
Approval Node
↓
interrupt()
↓
PAUSED
```

At this point:

Graph Status = Waiting

The application can return information to the UI:

"Approval required"

The user then chooses:

Approve

or:

Reject

32. Human Input

Human input can be something like:

Approve
Reject

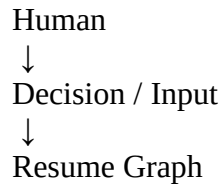
or:

```
{
  "approved": true
}
```

or even a modification:

```
{
  "amount": 4500
}
```

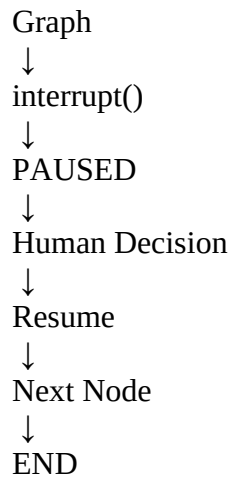
The important thing is:



33. Resume Execution

After the human provides the required input, the workflow can resume.

Conceptually:



This is different from starting a completely new workflow.

The important idea is:

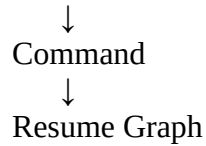
Resume means continuing from the persisted workflow state rather than starting the workflow from scratch.

34. Command

Command is used for controlling graph execution in situations such as resuming an interrupted graph and, depending on the use case, combining control-flow/state-update information.

Conceptually:

Human Decision



A conceptual example:

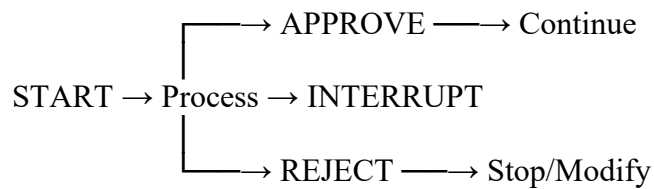
```
from langgraph.types import Command
```

```
Command(  
    resume="approve"  
)
```

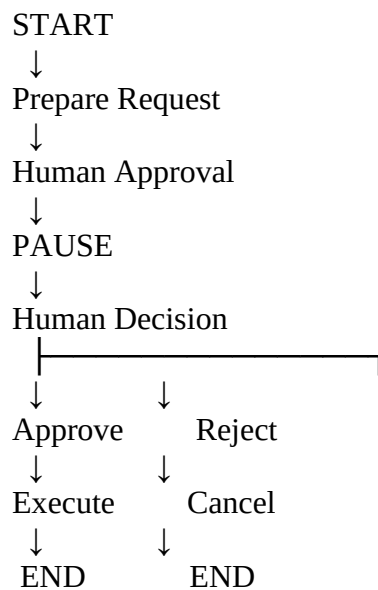
The exact command structure depends on what the graph is expecting.

35. Approve / Reject Workflow

A common HITL architecture:



Detailed:



36. Modify State Through Human Input

HITL doesn't have to be only:

YES / NO

A human can provide corrected information.

Example:

System:

Amount = ₹50,000

Human:

Change amount to ₹45,000

Flow:

Generate Data



Interrupt



Human modifies



State update

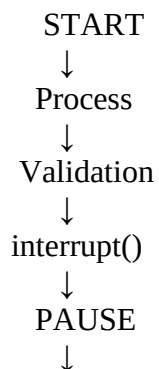


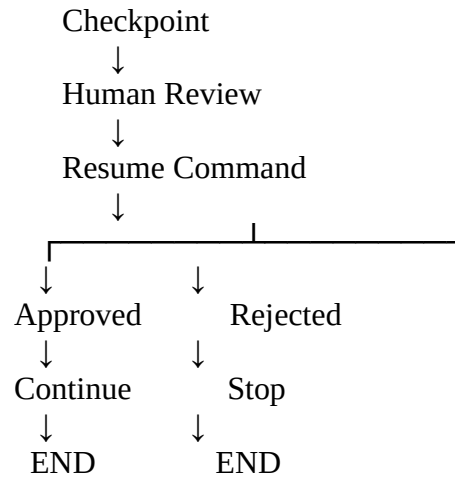
Continue

This is powerful for review workflows.

37. Complete HITL Architecture

Remember:

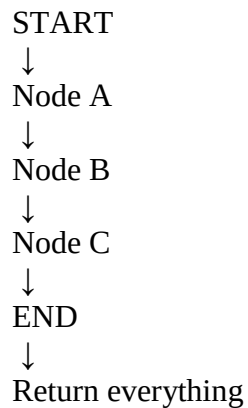




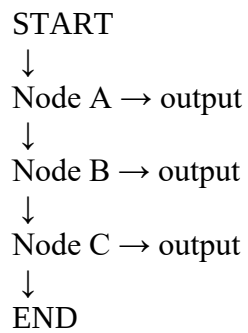
38. What is Streaming?

Streaming means receiving execution output progressively instead of waiting for the entire graph to finish.

Without streaming:



With streaming:



The application can observe progress while the workflow is running.

39. Why Streaming?

Streaming is useful for:

- Chat applications
- LLM responses
- Progress indicators
- Long-running workflows
- Real-time UI updates
- Debugging
- Monitoring

Instead of:

Please wait...
Please wait...
Please wait...
Complete!

you can show:

Processing...
Analyzing...
Generating...
Completed

40. stream()

stream() is used for synchronous streaming.

Conceptually:

```
for chunk in graph.stream(input_state):  
    print(chunk)
```

Instead of receiving one final result:

```
result = graph.invoke(...)
```

you receive execution output progressively.

Conceptually:

```
stream()
↓
Chunk 1
↓
Chunk 2
↓
Chunk 3
↓
...
```

41. astream()

astream() is the asynchronous version of streaming.

Conceptually:

```
async for chunk in graph.astream(input_state):
    print(chunk)
```

Comparison:

Method	Purpose
invoke()	Synchronous complete execution
ainvoke()	Asynchronous complete execution
stream()	Synchronous streaming
astream()	Asynchronous streaming

Easy memory trick

invoke → result

ainvoke → async result

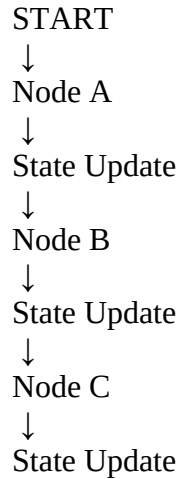
stream → chunks

astream → async chunks

42. Streaming State Updates

Streaming can expose state updates as nodes execute.

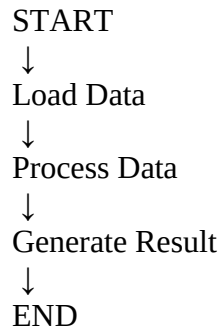
Example:



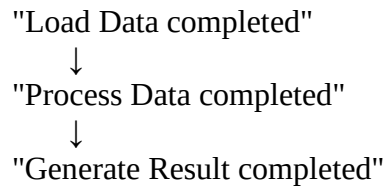
The application can observe these changes progressively.

43. Streaming Node Execution

Suppose:



Streaming can allow the application to observe progress:



This is useful for UI progress indicators.

44. Streaming Messages

For LLM-based applications, streaming can expose model-generated content progressively.

Example:

LLM starts

↓

"Lang"

↓

"LangGraph"

↓

"LangGraph is"

↓

"LangGraph is a framework..."

This creates the familiar typing effect in chat applications.

45. Streaming Events

Depending on the streaming mode/API being used, you can observe different kinds of execution information.

Conceptually:

Graph Started

↓

Node Started

↓

Node Output

↓

Node Finished

↓

LLM Output

↓

Graph Finished

This can be useful for:

- Monitoring
 - Debugging
 - UI updates
 - Logging
 - Observability
-

46. Progress Updates

For long workflows:

START



Step 1



Step 2



Step 3



Step 4



END

The UI can display:

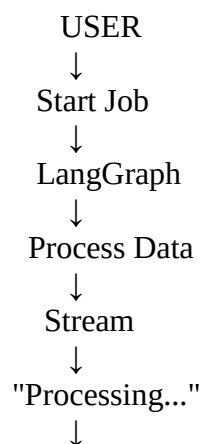
- ✓ Step 1 completed
- ✓ Step 2 completed
- Step 3 running
- Step 4 waiting

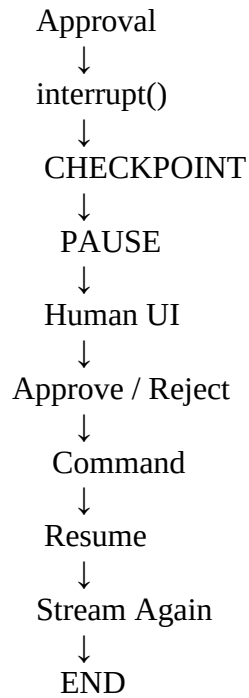
Streaming enables applications to receive incremental execution information instead of waiting for the entire workflow.

47. HITL + Persistence + Streaming

These three concepts work extremely well together.

Imagine an approval application:





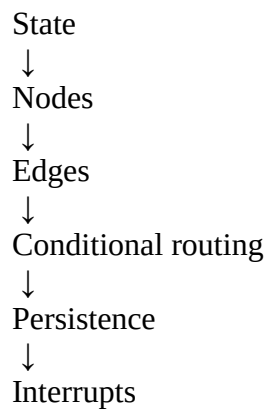
This is a very realistic architecture for production workflow applications.

Module 9 — Subgraphs, Parallelism & Advanced Graph Patterns

Goal

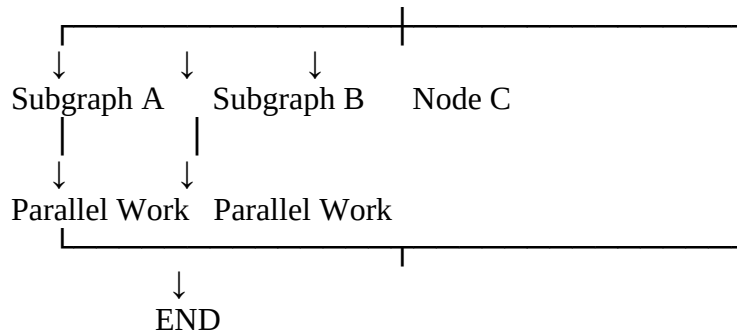
Learn how to design **large, modular, reusable, and parallel LangGraph applications.**

So far, you have learned:



Now we move toward larger workflows:



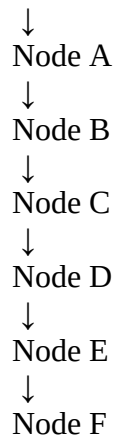


1. What is a Subgraph?

A **subgraph** is a graph embedded inside another graph.

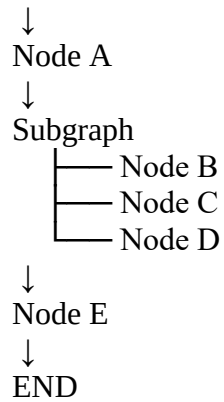
Instead of putting every node into one huge graph:

Main Graph



you can group related nodes into a separate graph:

Main Graph



Simple definition

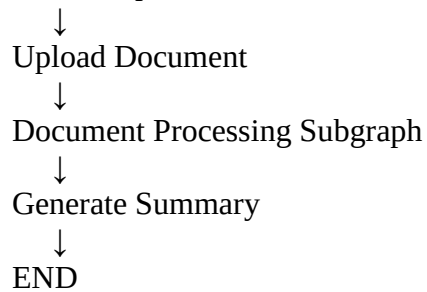
A subgraph is a graph used as a component inside another graph, allowing complex workflows to be divided into smaller workflows.

2. Why Use Subgraphs?

Subgraphs are useful for **modularity**.

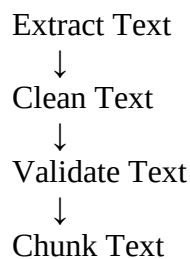
Imagine a document-processing application:

Main Graph



The document-processing part might itself contain:

Document Subgraph



This keeps the main graph easier to understand.

Benefits

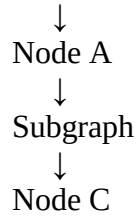
- Modular architecture
- Reusable workflows
- Easier testing
- Easier debugging
- Separation of responsibilities
- Smaller graphs
- Better maintainability

3. Parent Graph

The graph containing another graph is called the **parent graph**.

Example:

Parent Graph

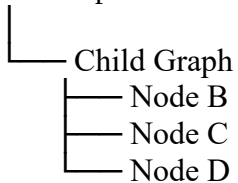


The parent graph controls the overall workflow.

4. Child Graph

The graph embedded inside the parent graph is the **child graph** or subgraph.

Parent Graph

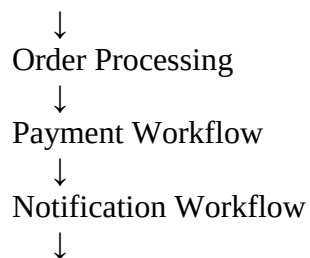


5. Nested Workflows

A workflow can contain another workflow.

For example:

Main Workflow

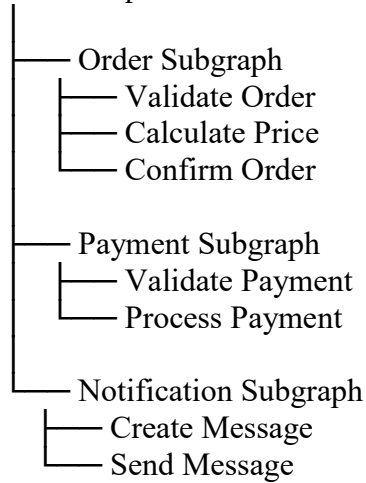


END

Each internal workflow can itself contain multiple nodes.

This creates a hierarchy:

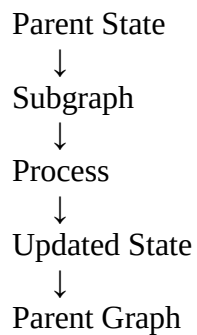
Main Graph



6. Sharing State Between Parent and Subgraph

A subgraph may work with information from the parent graph.

Conceptually:



Example:

Parent State:

```
{
  "document": "...",
  "status": "processing"
}
```

The subgraph processes the document and may produce:

```
{  
  "extracted_text": "...",  
  "status": "completed"  
}
```

The exact state-sharing approach depends on how the parent and subgraph schemas are designed.

7. Isolated State

A subgraph can also have its own internal state.

For example:

Parent State

```
{  
  document  
  status  
}
```

Subgraph:

Child State

```
{  
  text  
  chunks  
  validation_result  
}
```

This provides separation between:

Parent concerns

↓

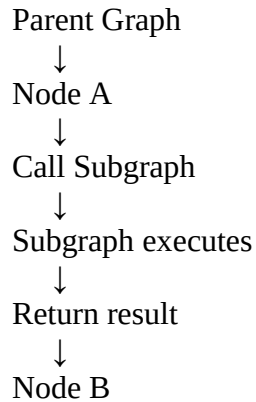
Child concerns

Important interview point

Subgraphs can be designed to either communicate through shared state/schema fields or maintain their own internal state, depending on how the graphs are composed.

8. Calling a Subgraph from a Node

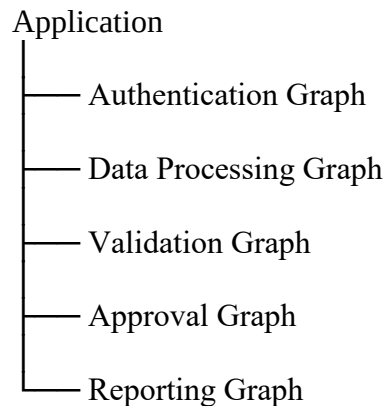
Conceptually, a node can invoke another graph.



Think of a subgraph as a reusable workflow component.

9. Subgraph Composition

Large applications can be composed from smaller graphs:

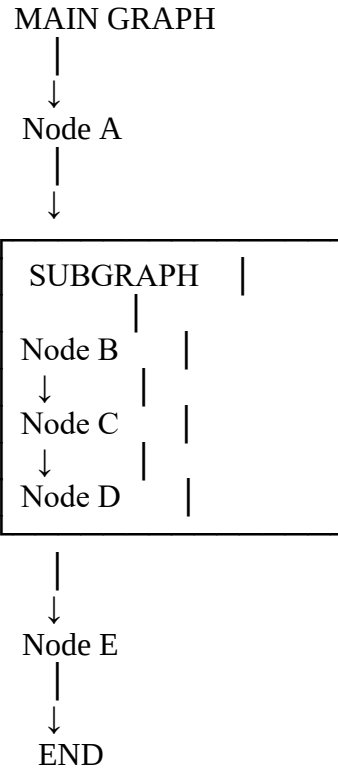


This is similar to software engineering principles such as:

- Modularity
 - Separation of concerns
 - Reusability
 - Composition
-

10. Subgraph Architecture

A typical structure:



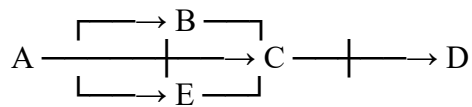
11. What is Parallelism?

Parallelism means executing independent parts of a workflow concurrently rather than waiting for one to finish before starting another.

Sequential:



Parallel:



If B, C, and E are independent, they can be executed concurrently.

12. Why Parallel Execution?

Suppose you need to process three independent tasks:

Task A = 5 seconds

Task B = 4 seconds

Task C = 3 seconds

Sequential:

$$5 + 4 + 3 = 12 \text{ seconds}$$

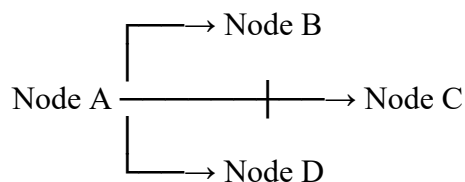
Conceptually, parallel execution can approach:

$$\max(5, 4, 3) = 5 \text{ seconds}$$

Actual performance depends on the runtime, model/API latency, resource availability, and workload.

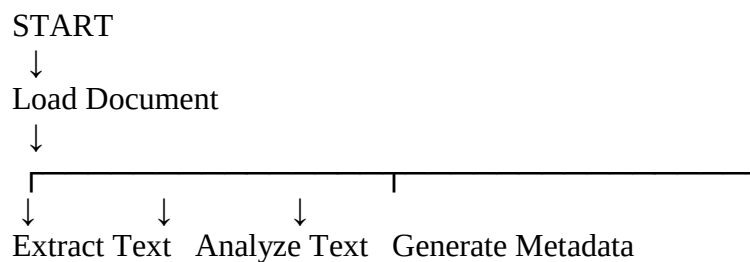
13. Fan-Out

Fan-out means one node branches into multiple parallel paths.



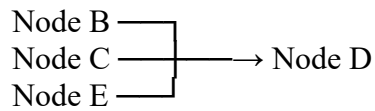
This is called **fan-out** because execution spreads outward.

Example:

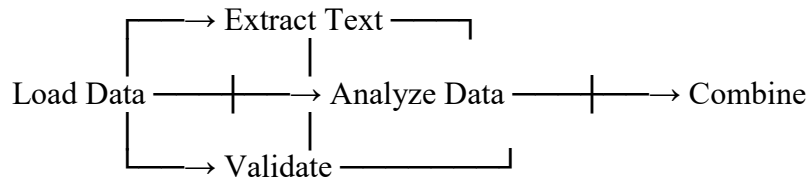


14. Fan-In

Fan-in means multiple branches converge into a later node.



Example:



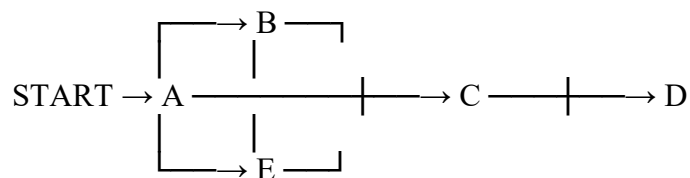
Easy memory trick

Fan-out → one → many

Fan-in → many → one

15. Fan-Out + Fan-In

This is one of the most important parallel workflow patterns.



Where:

A = distribute work

B/C/E = independent work

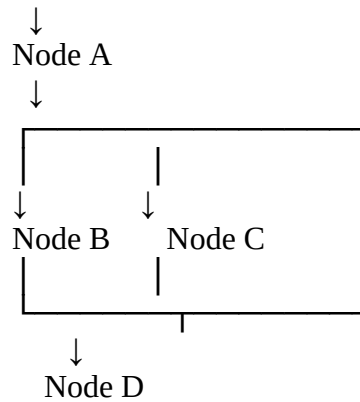
D = combine results

16. Concurrent Execution

If multiple nodes are ready and independent, LangGraph can execute them concurrently according to its execution model.

Conceptually:

START



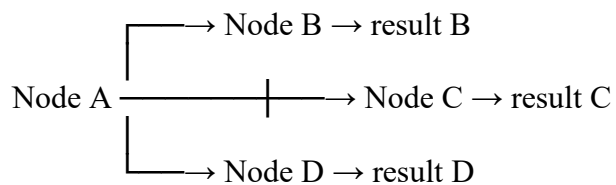
The important idea is:

Independent branches can execute without unnecessarily waiting for one another.

17. Reducers with Parallel Updates

Parallel nodes can produce updates to the same state field.

For example:



Now the graph needs to combine:

result B
result C
result D

Reducers help determine how multiple updates are combined.

For example:

Annotated[list, operator.add]

can conceptually mean:

Existing list

Prathamesh Arvind Jadhav

```
+  
Node B result  
+  
Node C result  
+  
Node D result
```

Result:

```
[  
  result_B,  
  result_C,  
  result_D  
]
```

18. Handling Parallel State Conflicts

Suppose two parallel nodes update:

status

Node B:

status = "approved"

Node C:

status = "rejected"

What should the final state be?

This is a **state update conflict**.

You must design the state schema/reducer so that concurrent updates are handled correctly.

Instead of having both nodes overwrite one field, you might store separate results:

```
{  
  "approval_result": ...,  
  "validation_result": ...  
}
```

or use an appropriate reducer when multiple updates need to be combined.

Interview point

Parallel execution makes state-update design important. If multiple branches write to the same state key, the reducer/schema must define how those updates are combined.

19. Sequential Pattern

The simplest workflow:

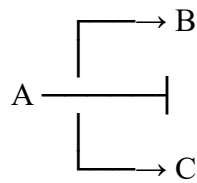


Use when:

- Steps depend on previous results
 - Order matters
 - No useful concurrency exists
-

20. Branch Pattern

A workflow chooses between paths.

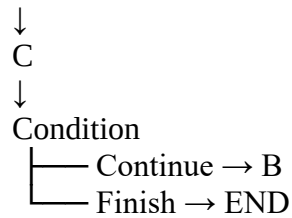


Usually implemented using conditional routing.

21. Loop Pattern

A workflow returns to an earlier node.



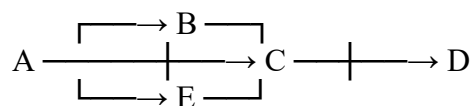


Useful for:

- Iterative processing
- Validation
- Retry logic
- Repeated transformation

22. Parallel Pattern

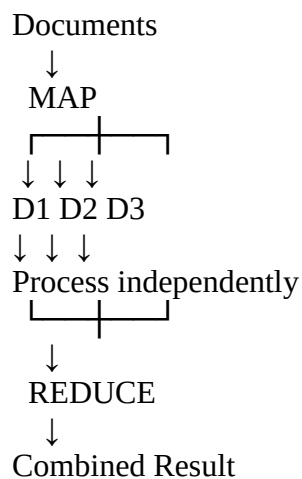
Independent work runs concurrently.



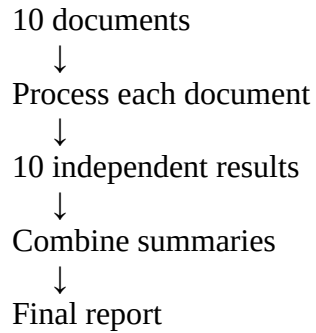
23. Map-Reduce Pattern

The **map-reduce pattern** is particularly useful for processing multiple items independently and then combining their results.

Example:

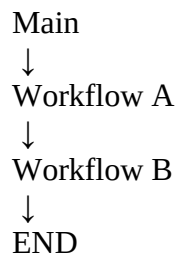


Example use case:



24. Nested Workflow Pattern

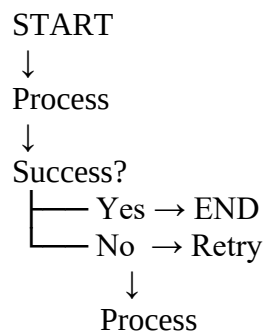
A workflow contains another workflow:



This is useful when each workflow represents a separate business responsibility.

25. Retry Pattern

A failed operation can be retried.



You should always have a termination condition:

Retry count < MAX_RETRIES

Otherwise:

Process



Fail



Retry



Fail



Retry



...

26. Human Approval Pattern

START



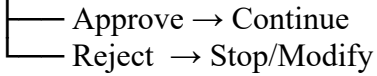
Process



interrupt()



Human Review



This combines:

- Nodes
- Conditional routing
- Interrupts
- Persistence
- State

27. Choosing the Right Pattern

Requirement	Pattern
Fixed ordered steps	Sequential
Choose between paths	Branch
Repeat processing	Loop
Independent tasks	Parallel

Requirement	Pattern
Many items + aggregation	Map-Reduce
Reusable complex workflow	Subgraph
Human decision	HITL
Temporary failure	Retry

Module 10 — Debugging, Testing, Production

Goal

Learn how to move from:

Learning LangGraph

to:

Reliable LangGraph Application

A production workflow needs:

Development
↓
Testing
↓
Debugging
↓
Observability
↓
Persistence
↓
Error Recovery
↓
Deployment
↓
Monitoring

28. Debugging Graph Execution

Debugging means finding why the graph didn't behave as expected.

Typical questions:

Why didn't Node B execute?

Why did the graph choose Node C?

Why is state incorrect?

Why did execution stop?

Why did an LLM node fail?

Why is the graph looping?

A useful debugging strategy is:

Input



State



Node



Output



Next Node



State Update

Inspect each stage.

29. State Inspection

Since state is central to LangGraph, inspecting state is one of the best debugging techniques.

Example:

Before Node:

```
{  
  "status": "pending"  
}
```

After Node:

```
{  
  "status": "approved"  
}
```

If the next node doesn't execute as expected, inspect:

State



Routing decision



Next node

30. State History

With persistence/checkpointing, you can inspect previous states.

Checkpoint 1



Checkpoint 2



Checkpoint 3



Checkpoint 4

This helps answer:

"At which point did my state become incorrect?"

31. Checkpoints

Checkpoints are valuable during debugging because they provide saved execution states.

Example:

START



Node A



Checkpoint 1



Node B



Checkpoint 2



Node C



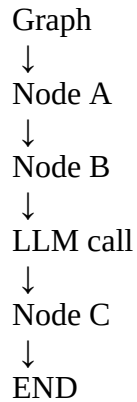
Checkpoint 3

If Node C produces an unexpected result, you can inspect the state leading into Node C.

32. Execution Tracing

Tracing records information about execution.

Conceptually:



A trace can show:

Node A → completed
Node B → completed
LLM → 1.4 seconds
Node C → failed

This is extremely useful in production.

33. Error Handling

Errors can happen at multiple levels.

Examples:

API failure
Database failure
LLM failure
Invalid state
Node exception
Timeout
External service unavailable

A production application should not simply crash without useful information.

34. Node-Level Errors

A specific node can fail.

START



Node A



Node B ☐



Node C

For example:

```
def process_data(state):  
    result = external_api_call()  
    return {"result": result}
```

If the API fails, Node B fails.

You should handle expected failures appropriately.

35. Graph-Level Errors

A graph-level problem can occur because of:

- Invalid graph configuration
- Unhandled node errors
- Recursion limit
- Invalid routing
- Invalid state updates
- External service failures

Conceptually:

Graph

├── Node A ✓

├── Node B ✓

├── Node C ☐

└── Execution failed

36. Retry Strategies

Not every error should cause permanent failure.

For temporary errors:

API Request



Failed



Retry



Success

Typical retry candidates:

- Temporary network failures
- Rate limits
- Transient API errors
- Temporary database failures

But don't blindly retry everything.

For example:

Invalid user input

usually shouldn't be retried repeatedly.

37. Timeouts

A workflow or external operation should not wait forever.

Without timeout:

Node



Waiting...



Waiting...



Waiting...



...

With timeout:

Node



Waiting



Timeout



Handle Failure

Timeouts are important for:

- APIs
- LLM calls
- Database calls
- Long-running operations

38. Recursion Limits

Loops and recursive graph paths can accidentally execute indefinitely.

Example:

A



B



A



B



A



...

LangGraph has recursion/execution limits to prevent unbounded execution.

Conceptually:

Loop



Iteration 1



Iteration 2



Iteration 3

↓
...
↓
Limit reached
↓
Stop/Error

39. Unit Testing Nodes

Nodes should ideally be testable independently.

Suppose:

```
def calculate_total(state):  
    ...
```

You can test:

Input State
↓
Node
↓
Output State

Example test idea:

Input:
price = 100
quantity = 3

Expected:
total = 300

This is easier than testing the entire graph for every small change.

40. Testing State Updates

Test whether a node produces the expected state update.

Input:

```
{  
    "count": 5
```

```
}
```

Node:

increment

Expected:

```
{  
  "count": 6  
}
```

41. Testing Routing Functions

Conditional routing should also be tested independently.

Example:

```
status = "approved"  
  ↓  
Expected route = "success"
```

And:

```
status = "rejected"  
  ↓  
Expected route = "failure"
```

This helps catch workflow-control bugs.

42. Testing Complete Graphs

After unit testing individual nodes, test the complete workflow:

```
Input  
  ↓  
Node A  
  ↓  
Node B  
  ↓  
Conditional Route  
  ↓  
Node C
```

↓
END

Verify:

- Final state
- Correct node path
- Correct routing
- Error behavior
- Expected termination

43. Mocking LLM Responses

LLMs are:

- Expensive
- Non-deterministic
- Slow
- Dependent on external services

Therefore, unit tests should often mock model responses.

Instead of:

Test
↓
Real LLM API
↓
Response

you can use:

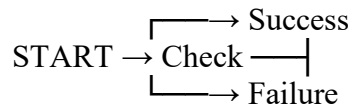
Test
↓
Mock LLM
↓
Fixed Response

This makes tests:

- Faster
- Cheaper
- More deterministic

44. Testing Conditional Paths

Suppose:



You should test both:

Input A → Success

Input B → Failure

Don't test only the happy path.

45. Testing Failure Scenarios

Production-quality tests should also cover:

LLM failure

API failure

Invalid state

Timeout

Database failure

Maximum retries

Unexpected model output

A good test strategy is:

Happy Path

+

Edge Cases

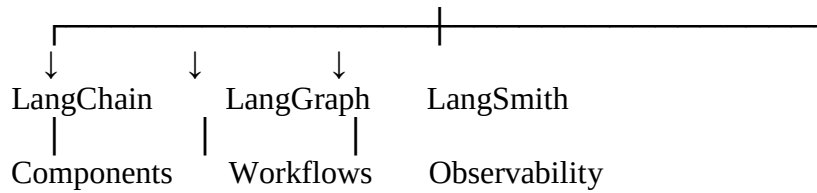
+

Failure Cases

46. LangGraph Ecosystem

You should understand how the major LangChain ecosystem components relate.

LangChain Ecosystem
|



47. LangChain

LangChain provides components for building LLM applications.

Examples include:

- Chat models
- Prompt templates
- Structured output
- Tools/integrations
- Retrievers
- Document processing

LangGraph can use LangChain components inside its nodes.

Think:

LangChain
↓
LLM / Prompt / Retriever
↓
LangGraph Node
↓
Graph Workflow

48. LangGraph

LangGraph focuses on **stateful graph-based workflows**.

Its core concepts include:

State
Nodes
Edges
Conditional Routing
Persistence
Checkpoints
Interrupts

Subgraphs
Streaming

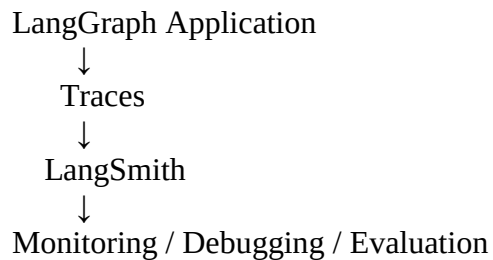
Think:

LangChain gives you components; LangGraph gives you a way to orchestrate stateful workflows using those components.

49. LangSmith

LangSmith is used for observability and evaluation of LLM applications and workflows.

Conceptually:



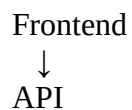
It can help you understand:

- What executed
 - Which node ran
 - Model calls
 - Latency
 - Errors
 - Inputs/outputs
 - Execution traces
-

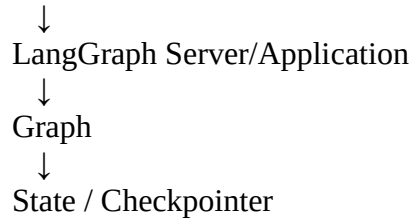
50. LangGraph Platform / Server Concepts

LangGraph applications can be exposed as services so that external applications can interact with workflows through APIs.

Conceptually:



Prathamesh Arvind Jadhav



This allows applications to treat a graph as a backend workflow service.

The exact deployment/platform products and APIs can change over time, so for implementation work you should follow the current LangGraph documentation.

51. API-Based Graph Execution

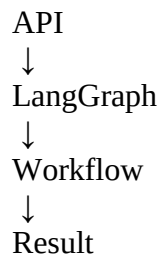
A frontend might send:

POST /workflow

with:

```
{  
  "message": "Process this document"  
}
```

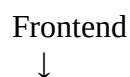
Backend:

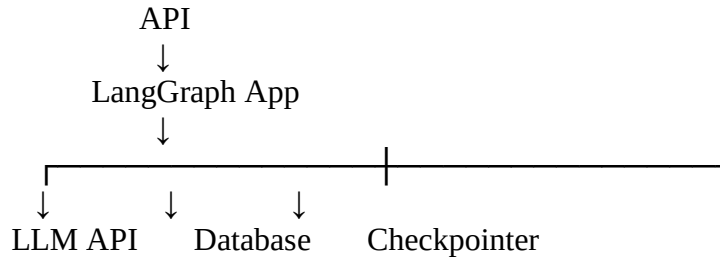


This allows LangGraph to become part of a larger web application.

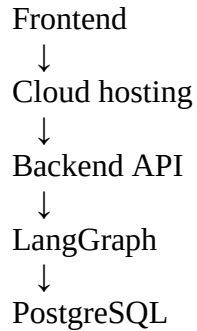
52. Deployment Concepts

A production architecture could look like:





Possible infrastructure:



53. Observability

Observability means understanding what your application is doing while it runs.

You want visibility into:

What happened?
Why did it happen?
How long did it take?
Where did it fail?
What state was present?

Important signals:

- Logs
- Traces
- Metrics
- Errors
- Latency

54. Monitoring

Monitoring means continuously checking application health and behavior.

For example:

Requests: 10,000
Success: 9,700
Failures: 300
Average latency: 2.3 sec

For LangGraph applications, useful metrics include:

- Execution count
 - Failure rate
 - Node latency
 - LLM latency
 - Token usage
 - Retry count
 - Timeout count
-

55. Logging

Logs help you understand what happened.

Example:

```
INFO Graph started
INFO Node validate started
INFO Node validate completed
INFO Node process started
ERROR API request failed
INFO Retrying...
INFO Graph completed
```

Avoid logging sensitive information such as:

- API keys
 - Passwords
 - Access tokens
 - Personal data unnecessarily
-

56. Persistence in Production

Production workflows often need durable state:

LangGraph



Checkpointers



PostgreSQL / supported durable storage

This enables:

- Recovery
 - Long-running workflows
 - HITL
 - State history
 - Resuming execution
-

57. Scalability

A production application may have:

1 user



10 users



1,000 users



100,000 users

You need to consider:

- Concurrent executions
 - Database capacity
 - LLM rate limits
 - API limits
 - Worker architecture
 - Connection pools
 - State storage
-

58. Concurrency

If 100 users execute workflows simultaneously:

User A → Graph A

User B → Graph B

User C $\rightarrow$ Graph C

...

User 100 $\rightarrow$ Graph 100

Their state must remain properly isolated.

Thread IDs and persistent state management become important.

59. Error Recovery

Production systems should have recovery strategies:

Failure



Identify error



Is it retryable?



Yes $\rightarrow$ Retry



No $\rightarrow$ Handle / Stop

For long-running workflows:

Failure



Checkpoint exists



Recover state



Resume

60. Authentication

If your LangGraph application is exposed through an API:

Frontend



Authentication



Authorization



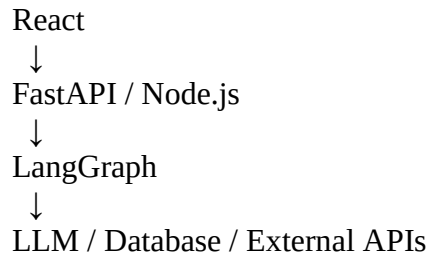
LangGraph API

You should verify:

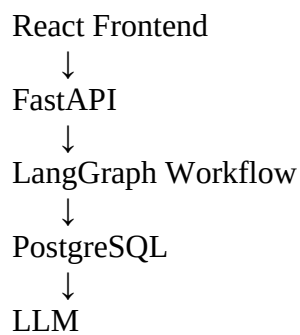
- Who is the user?
 - What are they allowed to access?
 - Which thread/state belongs to them?
-

61. API Integration

LangGraph can be part of a larger application:



For example:



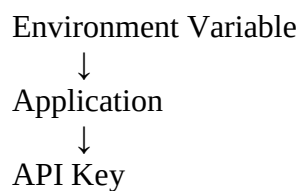
62. Environment Variables

Don't hard-code configuration:

Bad:

```
API_KEY = "my-secret-key"
```

Better:



Typical environment variables:

OPENAI_API_KEY
DATABASE_URL
LANGSMITH_API_KEY

63. Secrets Management

Secrets should be protected.

Examples:

- API keys
- Database passwords
- JWT secrets
- Cloud credentials

Use:

Environment variables
+
Secret management systems

rather than committing secrets to Git.

64. Database Integration

Production LangGraph applications commonly interact with databases.

Architecture:

LangGraph
↓
Database Layer
↓
PostgreSQL

Database may store:

- Application data
- User data
- Workflow-related data
- Checkpoints

- Long-term application memory

Keep the distinction clear between **business/application data** and **workflow checkpoint data**.

65. Performance Optimization

Potential optimization areas:

1. Parallelize independent work

A
↓
B + C + D

instead of:

A
↓
B
↓
C
↓
D

2. Avoid unnecessary LLM calls

Don't call an LLM when deterministic code can solve the problem.

3. Use appropriate model sizes

Simple task:

Smaller/faster model

Complex task:

More capable model

4. Cache expensive operations

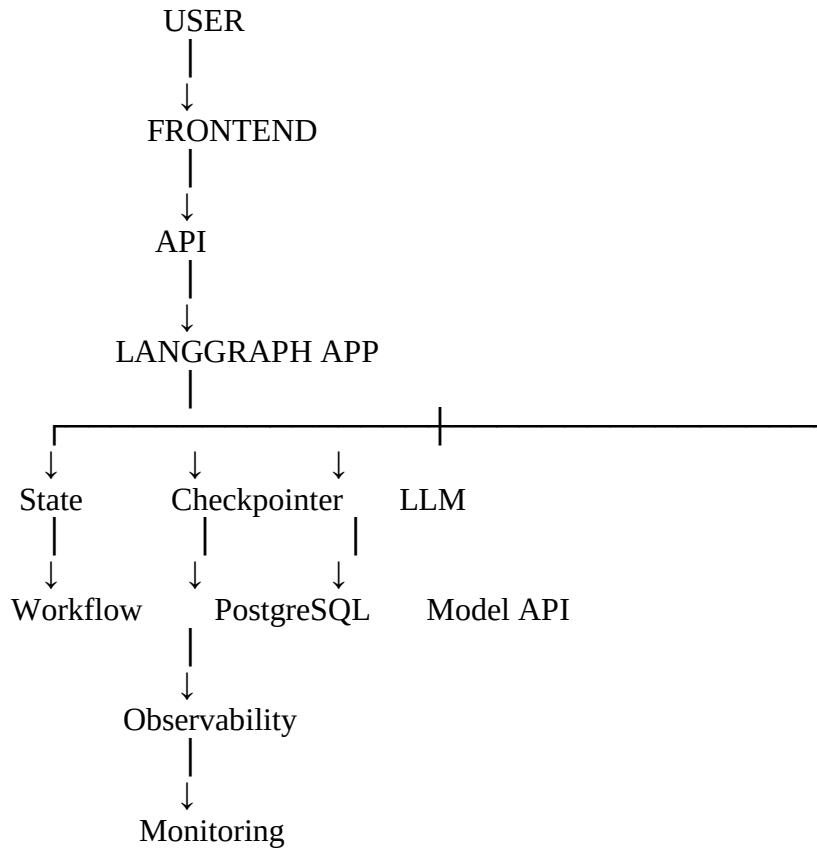
Expensive computation

↓
Cache
↓

Reuse

66. Production Architecture

A simplified production LangGraph architecture:



☐ Interview Preparation

Now you should be able to explain LangGraph from beginning to production.

67. LangGraph vs LangChain

This is one of the **most important interview questions**.

LangChain	LangGraph
Component/framework ecosystem	Graph-based workflow framework
Models, prompts, retrievers, tools etc.	Nodes, edges, state, workflows

LangChain	LangGraph
Useful for building LLM application components	Useful for orchestrating stateful workflows
Can be relatively straightforward	Designed for complex stateful execution
Less focused on graph state management	Strong state management
Basic sequential flows are easy	Branching/loops/parallel workflows are natural
Can be used inside LangGraph	Can use LangChain components

Interview answer

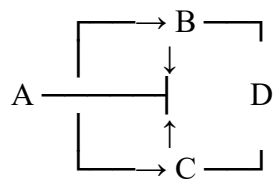
LangChain provides components and abstractions for building LLM applications, while LangGraph focuses on orchestrating those components into stateful, graph-based workflows with features such as conditional routing, persistence, checkpointing, interrupts, and streaming.

68. LangGraph vs Traditional Workflow Engines

Traditional workflow:

$A \rightarrow B \rightarrow C \rightarrow D$

LangGraph can represent:



It naturally supports:

- State
- Cycles
- Conditional routing
- Human interruptions
- LLM-based processing
- Streaming
- Checkpointing

Traditional workflow engines can also support branching, retries, persistence, and complex orchestration, so the distinction is not that traditional engines "cannot" do these things. The key

difference is that LangGraph is particularly oriented toward **stateful LLM/application workflows**.

69. State vs Memory

State

Current workflow information

Memory

Information intentionally retained for future use

Example:

State:

current_step = "approval"

Memory:

user_preference = "Python"

70. Node vs Edge

Node

Performs work.

Node



Process data

Edge

Determines where execution goes.

Node A



Edge



Node B

Easy interview answer

A node performs an operation, while an edge controls the transition between nodes.

71. Checkpoint vs State

State

Current workflow data:

```
{  
  "status": "processing"  
}
```

Checkpoint

Persisted snapshot of that state at an execution point.

Checkpoint

↓

State snapshot

Interview answer

State is the current data used by the workflow, while a checkpoint is a persisted snapshot of workflow state at a particular execution point.

72. invoke() vs stream()

invoke()	stream()
Runs graph and returns result	Produces execution output progressively
Good for complete result	Good for real-time updates
Wait for execution result	Observe execution as it happens

And:

invoke() → synchronous result

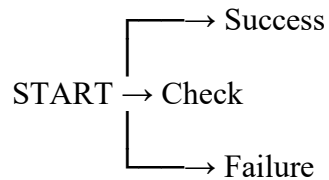
ainvoke() → asynchronous result

stream() → synchronous streaming

astream() → asynchronous streaming

73. Conditional Edges

Conditional edges choose the next node based on some condition.

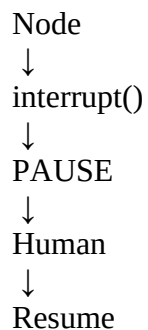


The routing decision can depend on:

- State
 - Validation result
 - Model output
 - Application logic
-

74. Interrupts

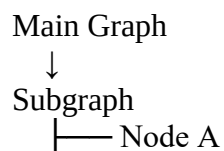
An interrupt pauses execution:

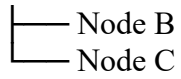


Usually paired with persistence/checkpointing when the pause needs to survive across requests or application execution.

75. Subgraphs

Subgraphs help organize complex applications:





Main Graph

Main benefits:

Modularity

Reusability

Separation of concerns

Testing

Maintainability